BACKEND SDE

COMPLETE INTERVIEW GUIDE

50 Essential Questions • Systems Design • Architecture • Databases

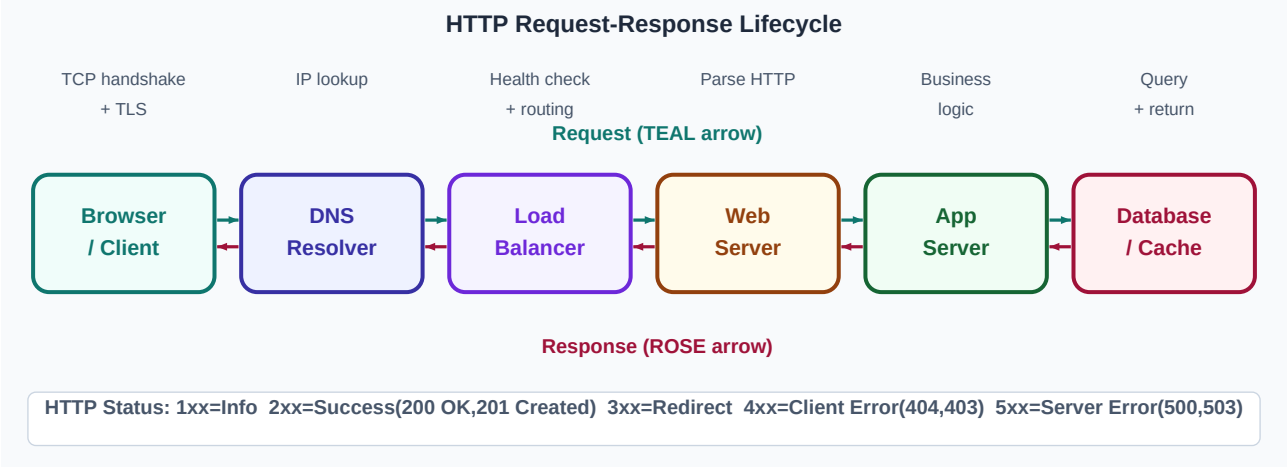
---

LEVEL 1: FUNDAMENTALS

HTTP, REST, Databases, Caching, Docker, CI/CD

Q1 - Q18

Q1 What is HTTP? Explain request-response lifecycle, methods, status codes, and headers.



HTTP lifecycle: Client -> DNS -> Load Balancer -> Web Server -> App Server -> Database. Request (TEAL) and Response (ROSE) arrows show data flow.

HTTP FUNDAMENTALS

HTTP (HyperText Transfer Protocol) is a stateless application-layer protocol for transmitting data between clients and servers. Every web request follows a request-response cycle.

HTTP Methods (Verbs):

- GET: retrieve data, safe and idempotent, no request body.
- POST: create resource, not idempotent, has request body.
- PUT: replace entire resource, idempotent.
- PATCH: partial update, not necessarily idempotent.

- DELETE: remove resource, idempotent.
- HEAD: like GET but response body omitted (check existence/headers).
- OPTIONS: query allowed methods for a resource (used in CORS preflight).

STATUS CODES

```
2xx Success:
```

```
200 OK — standard success
```

```
201 Created — resource created (after POST)
```

```
204 No Content — success, no body (after DELETE)
```

```
3xx Redirection:
```

```
301 Moved Permanently — SEO-safe redirect
```

```
302 Found — temporary redirect
```

```
304 Not Modified — use cached version
```

```
4xx Client Error:
```

```
400 Bad Request — invalid input
```

```
401 Unauthorized — authentication required
```

```
403 Forbidden — authenticated but no permission
```

```
404 Not Found — resource doesn't exist
```

```
429 Too Many Req. — rate limit exceeded
```

```
5xx Server Error:
```

```
500 Internal Server Error — unhandled exception
```

```
502 Bad Gateway — upstream failure
```

```
503 Service Unavailable — server overloaded
```

IMPORTANT HEADERS

- Content-Type: application/json — body format.
- Authorization: Bearer {token} — auth token.
- Cache-Control: max-age=3600 — caching directive.
- X-Request-ID: uuid — trace requests across services.
- Accept: application/json — client's preferred response format.
- CORS: Access-Control-Allow-Origin — which origins can access.

INTERVIEW TIP

401 vs 403 is a very common interview trick: 401=not authenticated (send credentials), 403=authenticated but not authorized (you have no permission). GET is idempotent, POST is not. Always use 201 for resource creation, not 200.

CHEAT SHEET ANCHOR

HTTP=stateless request-response. GET=read,POST=create,PUT=replace,PATCH=partial,DELETE=remove. 200=OK,201=Created,204=No Body,401=no auth,403=forbidden,404=missing,429=rate limit,500=server error. Idempotent=same result on repeat: GET,PUT,DELETE yes. POST no.

Q2 What is REST API? Explain its principles and how to design a RESTful API.

REST API Design — Resources, Methods, Status Codes

Method	Endpoint	Description	Status Code
GET	/users	List all users	200 OK
GET	/users/{id}	Get specific user	200 OK / 404
POST	/users	Create new user	201 Created
DELETE	/users/{id}	Delete user	204 No Content/404

Stateless
No session on server — each request has all info needed

Resource-based
URL = noun(/users), HTTP method = verb(GET/POST)

Uniform Interface
Standard HTTP methods, content negotiation(JS

Idempotent
GET,PUT,DELETE: same result on repeat calls. POST: not idempotent.

REST API: resources as nouns in URLs, HTTP methods as verbs. Stateless, uniform interface, resource-based design.

REST PRINCIPLES (Roy Fielding 2000)

- Stateless: Every request contains all necessary information. Server stores NO session state. Each request is independent.
- Client-Server: Separation of concerns — client handles UI, server handles data.
- Cacheable: Responses should declare if they're cacheable (Cache-Control header).
- Uniform Interface: Standard HTTP methods. Resources identified by URLs. Representation (JSON/XML) separate from resource.
- Layered System: Client doesn't know if it's talking to origin server or proxy.
- Code on Demand (optional): Server can send executable code (JavaScript).

REST API DESIGN BEST PRACTICES

```
# Good URL design
```

```
GET /users # list users
```

```
GET /users/123 # get user 123
```

```
POST /users # create user
PUT /users/123 # replace user 123
PATCH /users/123 # partial update
DELETE /users/123 # delete user 123
GET /users/123/orders # user's orders (nested resource)

# Filtering, sorting, pagination
GET /products?category=electronics&sort=price&order=asc&page=2&limit=20

# Versioning
GET /api/v1/users # URL versioning (most common)
GET /users Header: Accept: application/vnd.api+json;version=1

# Bad URL patterns to avoid
GET /getUser/123 # verb in URL — wrong
POST /users/create # redundant — POST means create
GET /users/123/deleteUser # wrong verb+noun combo
```

REST vs GraphQL vs gRPC

- REST: Simple, HTTP native, over-fetching/under-fetching issues. Best for: public APIs, simple CRUD.
- GraphQL: Client specifies exact fields needed, single endpoint. Best for: complex frontends with varied data needs, mobile (bandwidth sensitive).
- gRPC: Protocol Buffers binary format, HTTP/2, streaming. Best for: internal microservice communication, high-performance RPC.

INTERVIEW TIP

URL design rule: URLs = nouns (resources), HTTP methods = verbs. Never put verbs in URLs. Use plural nouns (/users not /user). Use nested resources for relationships (/users/123/orders). Always paginate list endpoints — never return unbounded results.

CHEAT SHEET ANCHOR

REST=stateless+client-server+cacheable+uniform interface. URLs=nouns(/users), HTTP methods=verbs. Versioning: /api/v1/. Pagination: ?page=2&limit=20. Filter: ?category=X&sort=price. REST vs GraphQL(flexible queries) vs gRPC(binary+streaming, internal services).

Q3 What is the difference between SQL and NoSQL databases? When to choose each?

SQL DATABASES (RELATIONAL)

SQL databases store data in structured tables with predefined schemas. Relationships enforced via foreign keys. ACID transactions guaranteed.

```
# Examples: PostgreSQL, MySQL, SQLite, Oracle
# Use when:
# - Data has clear relationships (users -> orders -> products)
# - ACID transactions required (banking, payments, inventory)
# - Complex queries with JOINs needed
# - Schema is stable and well-defined
```

```
# PostgreSQL example
CREATE TABLE users (
id SERIAL PRIMARY KEY,
email VARCHAR(255) UNIQUE NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
SELECT u.email, COUNT(o.id) as order_count
FROM users u
JOIN orders o ON u.id = o.user_id
GROUP BY u.email
HAVING COUNT(o.id) > 5;
```

NOSQL DATABASES

Document (MongoDB): JSON documents, flexible schema. Use for: catalogs, content, user profiles.

```
# MongoDB example
db.products.insertOne({
name: 'Laptop', price: 999,
specs: { ram: '16GB', storage: '512GB SSD' },
tags: ['electronics', 'computers']
})
db.products.find({ tags: 'electronics', price: { $lt: 1500 } })
```

Key-Value (Redis, DynamoDB): O(1) lookup. Use for: sessions, caching, rate limiting.

```
import redis
r = redis.Redis()
r.setex('session:abc123', 3600, json.dumps(user_data))
```

```
r.get('session:abc123')
```

Column-Family (Cassandra): high write throughput. Use for: IoT, time-series, activity feeds.

Graph (Neo4j): relationship traversal. Use for: social networks, fraud detection, recommendations.

DECISION FRAMEWORK

- Choose SQL: ACID needed, complex JOINs, stable schema, financial data.
- Choose NoSQL: horizontal scale, flexible schema, high write throughput, specific access patterns.
- Polyglot persistence: use multiple databases for different needs in the same system (PostgreSQL for transactions + Redis for caching + Elasticsearch for search).

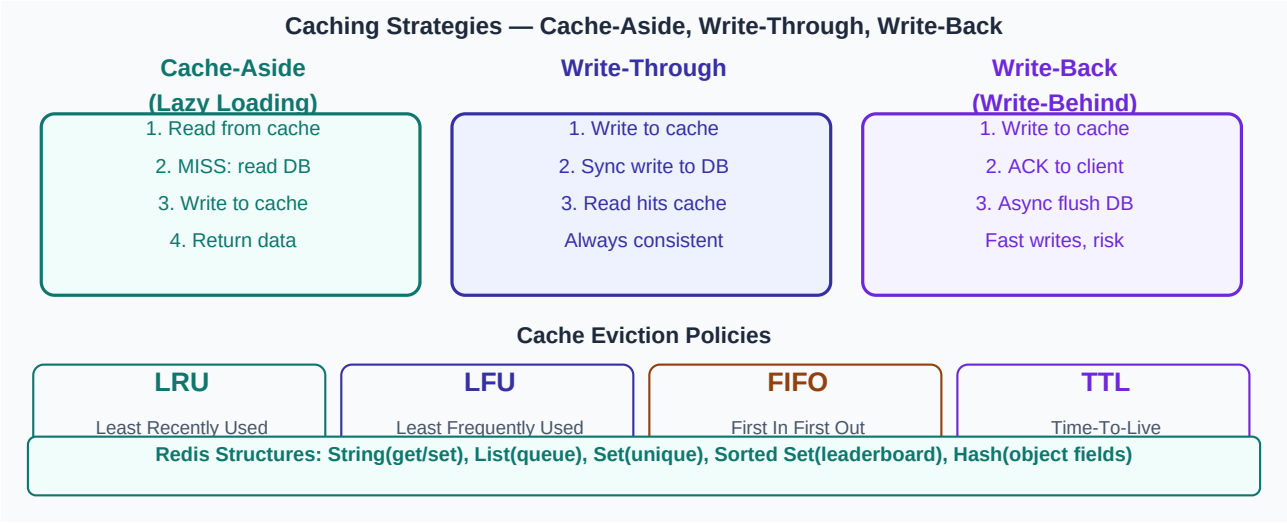
INTERVIEW TIP

The most important answer: most production systems use both (polyglot persistence). PostgreSQL for the main transactional data, Redis for caching/sessions, Elasticsearch for search. Don't frame it as SQL vs NoSQL — frame it as 'right tool for each use case'.

CHEAT SHEET ANCHOR

SQL=ACID+schema+JOINs, PostgreSQL/MySQL. NoSQL: Document(MongoDB, flexible schema), Key-Value(Redis, O(1)), Column(Cassandra, write throughput), Graph(Neo4j, relationships). Choose SQL for: transactions, relations. NoSQL for: scale, flexibility. Production=polyglot.

Q4 What is caching? Explain cache strategies, eviction policies, and Redis data structures.



Caching strategies: Cache-Aside (lazy loading), Write-Through (sync write), Write-Back (async flush). Eviction: LRU, LFU, FIFO, TTL.

CACHING FUNDAMENTALS

Caching stores frequently accessed data in fast memory (Redis/Memcached) to reduce database load and response latency. The 80/20 rule: 20% of data accounts for 80% of reads.

```
import redis

r = redis.Redis(host='localhost', port=6379, db=0)

# Cache-Aside (Lazy Loading) pattern

def get_user(user_id):

cache_key = f'user:{user_id}'

cached = r.get(cache_key)

if cached:

return json.loads(cached) # cache HIT

user = db.query('SELECT * FROM users WHERE id=%s', user_id)

r.setex(cache_key, 3600, json.dumps(user)) # cache 1 hour

return user # cache MISS, populated
```

REDIS DATA STRUCTURES WITH USE CASES

```
# String — simple key-value, session storage

r.set('user:123:name', 'Alice')

r.setex('session:token', 86400, user_id)

# Hash — object fields, user profile

r.hset('user:123', mapping={'name':'Alice','email':'a@b.com'})

r.hget('user:123', 'email')

# List — queue, recent activity feed

r.lpush('notifications:123', json.dumps(notif))

r.lrange('notifications:123', 0, 9) # last 10

# Set — unique items, tags, followers

r.sadd('user:123:followers', 456, 789)

r.sismember('user:123:followers', 456) # O(1) check

# Sorted Set — leaderboard, priority queue

r.zadd('leaderboard', {'player1': 9500, 'player2': 8200})

r.zrevrange('leaderboard', 0, 9, withscores=True) # top 10

# Pub/Sub — real-time notifications

r.publish('notifications', json.dumps({'user': 123, 'msg': 'hello'}))
```

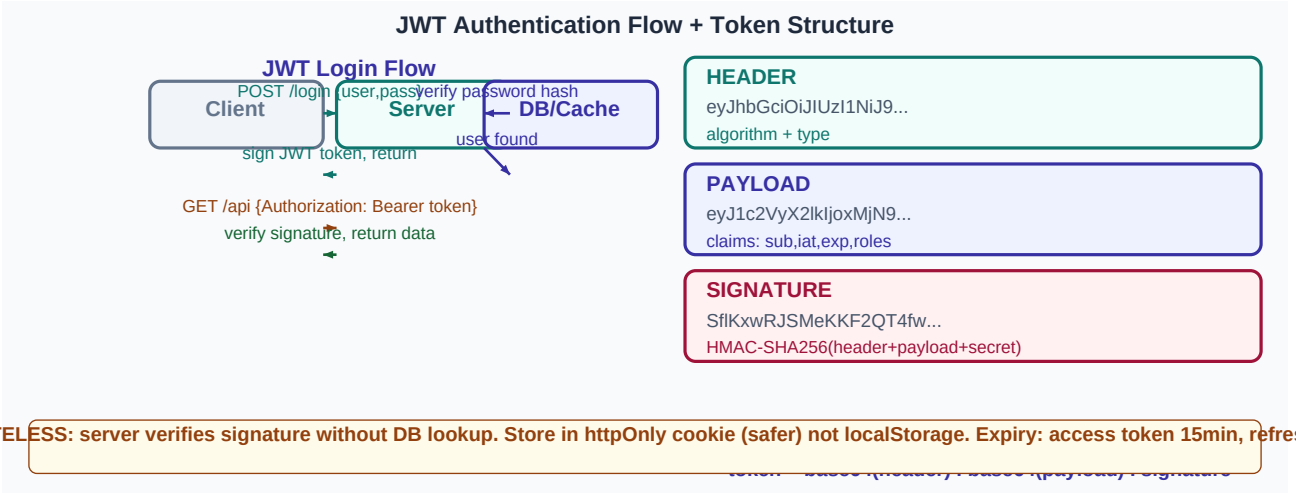
INTERVIEW TIP

Cache invalidation is the hardest problem. Three main issues: stale data (TTL helps), thundering herd (many cache misses at once — use lock/mutex to rebuild once), and cache stampede. Write-Through guarantees fresh cache but has write latency. Write-Back is fastest but risks data loss on crash.

CHEAT SHEET ANCHOR

Caching: Cache-Aside(lazy load, miss->DB->cache), Write-Through(write cache+DB sync), Write-Back(write cache, async DB). Eviction: LRU(least recently), LFU(least frequency), TTL(time expiry). Redis: String(session), Hash(object), List(feed), Set(unique), SortedSet(leaderboard), Pub/Sub.

Q5 Explain authentication vs authorization. Implement JWT-based authentication.



JWT flow: POST /login -> verify password -> sign token -> return. Subsequent: Bearer token in header -> verify signature -> serve. Token = header.payload.signature.

AUTHENTICATION vs AUTHORIZATION

- Authentication (authn): Who are you? Verifying identity. Login with username+password, OAuth, biometrics.
- Authorization (authz): What can you do? Verifying permissions. 'Alice can read posts but not delete them.'
- Session-based: server stores session, client has session ID in cookie. Stateful — server must check DB/cache per request.
- JWT-based: server signs a token, client sends it. Stateless — server verifies signature without DB lookup.

JWT IMPLEMENTATION IN PYTHON

```
import jwt, bcrypt
```

```
from datetime import datetime, timedelta
```

```
SECRET = 'super-secret-key'
```

```
# Register: hash password
def register(email, password):
hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
db.execute('INSERT INTO users(email,password_hash) VALUES(%s,%s)',
email, hashed.decode())
```

```
# Login: verify + issue JWT
def login(email, password):
user = db.fetchone('SELECT * FROM users WHERE email=%s', email)
if not user or not bcrypt.checkpw(password.encode(),
user['password_hash'].encode()):
raise ValueError('Invalid credentials')
payload = {
'sub': user['id'],
'email': user['email'],
'roles': user['roles'],
'iat': datetime.utcnow(),
'exp': datetime.utcnow() + timedelta(minutes=15),
}
access_token = jwt.encode(payload, SECRET, algorithm='HS256')
refresh_token = jwt.encode({'sub': user['id'], 'exp':
datetime.utcnow() + timedelta(days=7)}, SECRET, 'HS256')
return {'access_token': access_token, 'refresh_token': refresh_token}
```

```
# Middleware: verify JWT
def require_auth(f):
def wrapper(*args, **kwargs):
token = request.headers.get('Authorization','').split(' ')[-1]
try:
payload = jwt.decode(token, SECRET, algorithms=['HS256'])
g.user = payload
except jwt.ExpiredSignatureError:
return {'error': 'Token expired'}, 401
except jwt.InvalidTokenError:
```

```
return {'error': 'Invalid token'}, 401
return f(*args, **kwargs)
return wrapper
```

INTERVIEW TIP

Access token (15 min) + Refresh token (7 days) is the production pattern. Short-lived access tokens limit damage from theft. When access token expires, client uses refresh token to get a new access token without re-login. Store access token in memory, refresh token in httpOnly cookie (not localStorage — XSS vulnerable).

CHEAT SHEET ANCHOR

Authentication=who are you(verify identity). Authorization=what can you do(verify permissions). JWT=stateless: header.payload.signature, server verifies without DB. Access token=15min, Refresh token=7days. bcrypt for passwords. Store refresh in httpOnly cookie, access in memory.

Q6 What is an ORM? Explain the N+1 problem and how to fix it. Show SQLAlchemy examples.

ORM FUNDAMENTALS

ORM (Object-Relational Mapper) maps database tables to Python classes. Write Python code instead of raw SQL. SQLAlchemy is the most popular Python ORM.

```
from sqlalchemy import Column, Integer, String, ForeignKey
from sqlalchemy.orm import relationship, DeclarativeBase
class Base(DeclarativeBase): pass
class User(Base):
__tablename__ = 'users'
id = Column(Integer, primary_key=True)
name = Column(String(100), nullable=False)
email = Column(String(200), unique=True)
posts = relationship('Post', back_populates='author',
lazy='select') # lazy by default
class Post(Base):
__tablename__ = 'posts'
id = Column(Integer, primary_key=True)
title = Column(String(200))
```

```
author_id = Column(Integer, ForeignKey('users.id'))
author = relationship('User', back_populates='posts')
```

N+1 PROBLEM AND FIX

```
# N+1 PROBLEM: 1 query for posts + N queries for each author
posts = session.query(Post).all() # 1 query
for post in posts:
print(post.author.name) # N queries! (1 per post)
# For 100 posts = 101 total queries
```

```
# FIX 1: joinedload (single JOIN query)
from sqlalchemy.orm import joinedload
posts = session.query(Post).options(
joinedload(Post.author)
).all()
# Now: 1 query with JOIN — no extra queries in loop
```

```
# FIX 2: selectinload (separate IN query, better for large sets)
from sqlalchemy.orm import selectinload
posts = session.query(Post).options(
selectinload(Post.author)
).all()
# Generates: SELECT * FROM users WHERE id IN (1,2,3,...) — 2 queries total
```

```
# FIX 3: Raw SQL for complex queries
result = session.execute(text(
'SELECT p.title, u.name FROM posts p JOIN users u ON p.author_id=u.id'
))
```

INTERVIEW TIP

N+1 is one of the most common backend performance bugs. Always log SQL queries in development (SQLAlchemy: echo=True) and look for repeated queries with different IDs. The fix: always use eager loading (joinedload or selectinload) when you know you'll access related objects in a loop.

CHEAT SHEET ANCHOR

ORM=maps tables to classes, Python instead of SQL. SQLAlchemy=most popular Python ORM. N+1=1 query for N objects + N queries for relations=slow. Fix: joinedload(single JOIN query) or selectinload(separate IN query). Enable echo=True in dev to catch N+1.

Q7 What is CORS? How does it work and how do you configure it in a backend API?

WHAT IS CORS?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that restricts web pages from making requests to a different domain than the one that served the page. Backend must explicitly allow cross-origin requests.

- Same-origin: same protocol + domain + port. http://app.com == http://app.com. http://app.com != https://app.com (different protocol) != http://api.app.com (different subdomain).

CORS FLOW AND PREFLIGHT

```
# Simple Request (GET with safe headers): no preflight

# Browser sends:

GET /api/users Origin: http://frontend.com

# Server must respond:

Access-Control-Allow-Origin: http://frontend.com
```

```
# Preflight Request (OPTIONS before POST/PUT/DELETE or custom headers)

# Browser sends OPTIONS automatically:

OPTIONS /api/users

Origin: http://frontend.com

Access-Control-Request-Method: POST

Access-Control-Request-Headers: Content-Type, Authorization
```

```
# Server responds to preflight:

Access-Control-Allow-Origin: http://frontend.com

Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS

Access-Control-Allow-Headers: Content-Type, Authorization

Access-Control-Max-Age: 86400 # cache preflight 24 hours
```

```
# Flask CORS configuration

from flask_cors import CORS

app = Flask(__name__)

CORS(app, resources={
```

```
r'/api/*': {
'origins': ['http://localhost:3000', 'https://myapp.com'],
'methods': ['GET', 'POST', 'PUT', 'DELETE'],
'allow_headers': ['Content-Type', 'Authorization'],
'supports_credentials': True, # allow cookies
'max_age': 86400
}
})
```

COMMON CORS MISTAKES

✗ Never use Access-Control-Allow-Origin: * in production with credentials (cookies/auth headers). Must be specific origin.

✗ CORS is ONLY enforced by browsers, not curl/Postman. APIs tested with Postman always 'work' — only frontend fails.

✗ Forgetting to handle OPTIONS method on API endpoints — preflight fails with 405 Method Not Allowed.

INTERVIEW TIP

CORS is browser-enforced only — APIs always accept requests from curl/Postman/server-to-server. If CORS blocks your frontend: the browser is protecting you from cross-site requests. Fix: add proper CORS headers on the backend, never on the frontend.

CHEAT SHEET ANCHOR

CORS=browser blocks cross-origin requests unless server allows. Preflight=browser sends OPTIONS before POST/PUT with custom headers. Server responds with Allow-Origin+Allow-Methods+Allow-Headers. Never * with credentials. Flask: flask_cors CORS(app, origins=['...']).

Q8 What is rate limiting? Implement Token Bucket algorithm with Redis.

Rate Limiting Algorithms Comparison

Token Bucket	Leaky Bucket	Fixed Window Counter	Sliding Window Log
Tokens added at fixed rate	Requests queue at fixed rate	Count requests per window	Store timestamp per request
Request consumes 1 token	Queue has max capacity	Reset at window boundary	Count in rolling window

```
# Token Bucket in Redis (Lua script for atomicity)
local tokens = tonumber(redis.call('GET', key) or capacity)
if tokens >= 1 then redis.call('SET', key, tokens-1, 'EX', window)
  return 1  -- allowed
else return 0  -- rejected
end
```

Token Bucket: tokens=min(capacity, tokens + rate*elapsed)

Production: Nginx limit_req, Kong rate-limiting plugin, or Redis + Lua for custom. Key: user_id:minute -> count.

Response headers: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset (Unix timestamp)

Rate limiting algorithms: Token Bucket (flexible burst), Leaky Bucket (smooth output), Fixed Window (simple), Sliding Window Log (precise). Production: Redis + Lua for atomicity.

WHY RATE LIMITING?

- Prevent abuse: stop users from sending thousands of requests per second.
- Protect backend: prevent database overload, ensure fair resource sharing.
- Cost control: in cloud environments, API calls cost money.
- DDoS mitigation: throttle suspicious IP addresses.

TOKEN BUCKET IMPLEMENTATION

```
import redis, time
from functools import wraps
from flask import request, jsonify

r = redis.Redis()

def rate_limit(max_requests=100, window=60):
def decorator(f):
@wraps(f)
def wrapper(*args, **kwargs):
# Key by IP or user ID
identifier = request.headers.get('X-User-ID',
request.remote_addr)
key = f'rate_limit:{identifier}:{f.__name__}'

pipe = r.pipeline()
now = time.time()
window_start = now - window

# Sliding window log approach
pipe.zremrangebyscore(key, 0, window_start) # remove old
pipe.zcard(key) # count current
pipe.zadd(key, {str(now): now}) # add this request
pipe.expire(key, window)
results = pipe.execute()

request_count = results[1]
```

```
if request_count >= max_requests:
return jsonify({'error': 'Rate limit exceeded'}), 429
```

```
response = f(*args, **kwargs)
response.headers['X-RateLimit-Limit'] = max_requests
response.headers['X-RateLimit-Remaining'] = max_requests - request_count - 1
return response
return wrapper
return decorator
```

```
# Usage
@app.route('/api/messages', methods=['POST'])
@rate_limit(max_requests=10, window=60) # 10 req/min
def send_message():
return {'status': 'sent'}
```

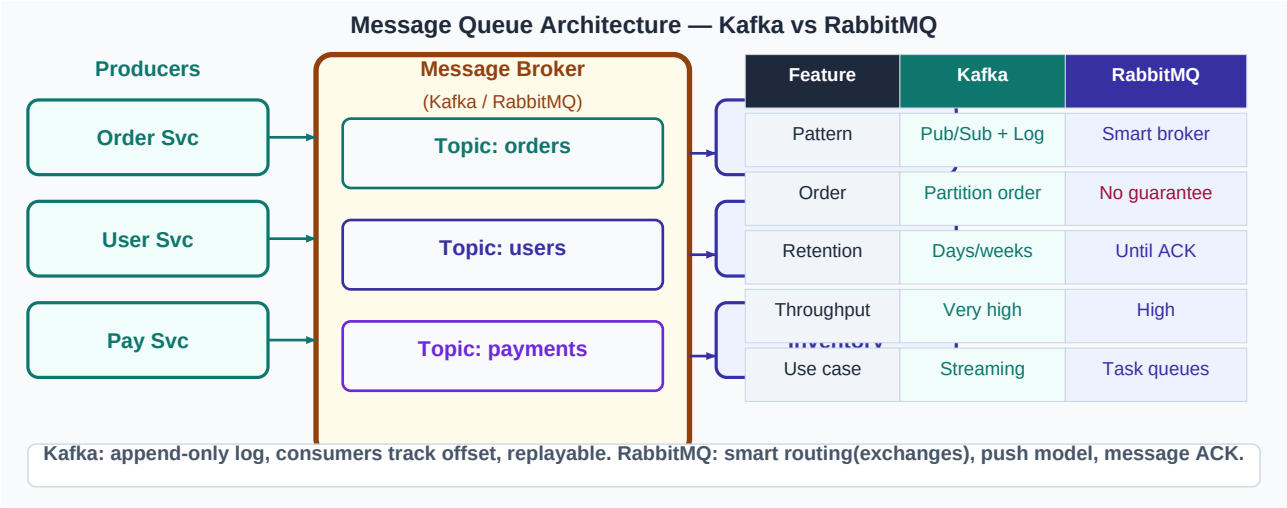
INTERVIEW TIP

Always add rate limit response headers: X-RateLimit-Limit (max allowed), X-RateLimit-Remaining (how many left), X-RateLimit-Reset (Unix timestamp when window resets). Use different limits for different tiers: free users get 100/hour, paid users get 10,000/hour.

CHEAT SHEET ANCHOR

Rate limiting: Token Bucket(flexible burst), Leaky Bucket(smooth output), Fixed Window(simple+boundary burst), Sliding Window(precise). Redis pipeline for atomicity. Response headers: X-RateLimit-Limit/Remaining/Reset. 429 status on exceeded. Key by user_id or IP.

Q9 What is a message queue? Explain use cases, and compare Kafka vs RabbitMQ.



Message queue: producers publish events to broker topics/queues, consumers process asynchronously. Kafka=append log+replay. RabbitMQ=smart routing+acknowledgements.

WHY MESSAGE QUEUES?

- Async processing: handle tasks in background (email sending, image resizing) without blocking the API response.
- Decoupling: Order service doesn't need to know Inventory/Notification services exist.
- Load leveling: 10,000 requests/sec can burst into queue, consumers process at sustainable 1,000/sec.
- Reliability: if consumer crashes, message stays in queue and retried.

KAFKA BASICS (Python)

```
from kafka import KafkaProducer, KafkaConsumer
import json

# Producer — publish event
producer = KafkaProducer(
bootstrap_servers=['kafka:9092'],
value_serializer=lambda v: json.dumps(v).encode()
)
producer.send('orders', {
'order_id': 12345,
'user_id': 99,
'amount': 149.99,
'items': [{'sku': 'LAPTOP-01', 'qty': 1}]
})
producer.flush()
```

```
# Consumer — process events
consumer = KafkaConsumer(
'orders',
bootstrap_servers=['kafka:9092'],
group_id='inventory-service', # consumer group
auto_offset_reset='earliest',
value_deserializer=lambda m: json.loads(m.decode())
)
for message in consumer:
order = message.value
inventory.decrement(order['items'])
consumer.commit() # mark as processed
```

RABBITMQ vs KAFKA

- Kafka: append-only log, messages retained for days. Consumers track their own offset. Best for: event streaming, audit logs, replayable events. High throughput.
- RabbitMQ: smart broker routes to queues via exchanges. Messages deleted after ACK. Best for: task queues, job processing, RPC-style patterns. Rich routing (topic, fanout, direct).
- Rule: Kafka for event streams where replay matters. RabbitMQ for task distribution where message routing matters.

INTERVIEW TIP

Dead Letter Queue (DLQ) is essential in production: if a message fails processing N times, it goes to a DLQ for manual inspection instead of being silently dropped. Always implement DLQ + alerting when messages accumulate there.

CHEAT SHEET ANCHOR

Message queue: async decoupling+load leveling+reliability. Kafka: append log, consumer tracks offset, replayable, high throughput. RabbitMQ: smart routing via exchanges, push model, message ACK/NACK. Use Kafka for streams/audit. RabbitMQ for task queues. DLQ for failed messages.

Q10 What is Docker? Explain containers vs VMs, Dockerfiles, and docker-compose.

CONTAINERS vs VIRTUAL MACHINES

Docker containers share the host OS kernel and isolate processes using namespaces and cgroups. VMs run complete guest OS on a hypervisor. Containers are lighter, faster, and portable.

```
# VM: 10-15 minute boot, 2-4GB RAM per VM, full OS
```

```
# Container: <1 second start, 50-200MB, shared kernel
```

```
# Docker architecture

# Docker Engine -> Docker Daemon -> Containers

# -> Images

# -> Networks

# -> Volumes
```

PRODUCTION DOCKERFILE

```
# Multi-stage build — smaller final image

FROM python:3.12-slim AS builder

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt
```

```
FROM python:3.12-slim AS runtime

WORKDIR /app

# Copy only what's needed from builder

COPY --from=builder /usr/local/lib/python3.12/site-packages
/usr/local/lib/python3.12/site-packages

COPY . .

# Non-root user for security

RUN useradd -m appuser && chown -R appuser:appuser /app

USER appuser

EXPOSE 8000

CMD ['gunicorn', '-w', '4', '-b', '0.0.0.0:8000', 'app:app']
```

DOCKER COMPOSE FOR LOCAL DEV

```
# docker-compose.yml

version: '3.8'

services:

api:

build: .

ports: ['8000:8000']

environment:

DATABASE_URL: postgresql://user:pass@db:5432/mydb
```

```
REDIS_URL: redis://redis:6379
depends_on: [db, redis]
volumes: ['./:/app'] # hot reload
---
db:
image: postgres:16
environment:
POSTGRES_USER: user
POSTGRES_PASSWORD: pass
POSTGRES_DB: mydb
volumes: ['pgdata:/var/lib/postgresql/data']
---
redis:
image: redis:7-alpine
---
volumes: {pgdata: {}}
```

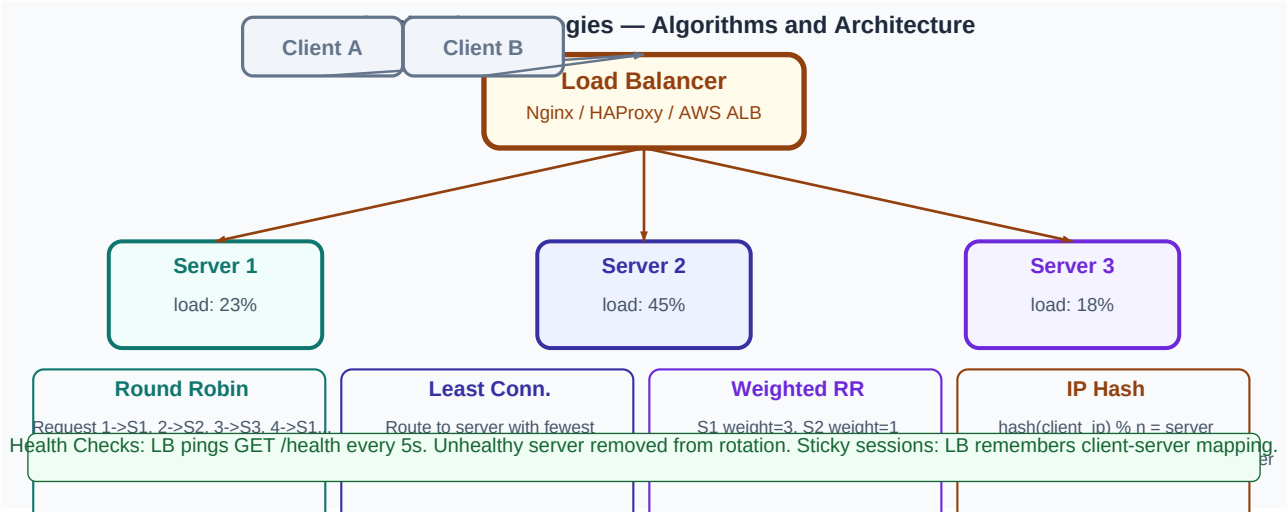
INTERVIEW TIP

Always use multi-stage builds in production Dockerfiles — the builder stage installs dependencies, the runtime stage only copies what's needed. This reduces image size from 1GB+ to ~100MB. Also: never run containers as root, never store secrets in Dockerfile ENV (use docker secrets or env files).

CHEAT SHEET ANCHOR

Docker=containers share OS kernel(lightweight). Container startup=<1s vs VM=10min. Multi-stage Dockerfile: builder installs deps, runtime stage copies only needed files(smaller image). docker-compose: local dev with DB+Redis+App. Run as non-root. Never put secrets in Dockerfile.

Q11 What is load balancing? Explain algorithms and how to configure Nginx.



Load balancer distributes traffic across servers. Algorithms: Round Robin, Least Connections, Weighted, IP Hash. Health checks remove unhealthy servers from rotation.

LOAD BALANCING ALGORITHMS

• Round Robin: requests cycle through servers sequentially. Simple, best when servers have equal capacity and requests take similar time.

• Least Connections: route to server with fewest active connections. Best when requests have variable processing time (short and long requests mixed).

• Weighted Round Robin: assign weights based on server capacity. Server with weight=3 gets 3x requests vs weight=1.

• IP Hash: hash client IP to consistently route to same server. Needed for sticky sessions without distributed session storage.

• Random with 2 choices: pick 2 random servers, choose the one with fewer connections. Better than pure random, nearly as good as least-connections.

NGINX LOAD BALANCER CONFIG

```
# nginx.conf

upstream backend {

least_conn; # or: round_robin (default), ip_hash

server app1:8000 weight=3; # 3x traffic

server app2:8000 weight=2;

server app3:8000 weight=1;

# Health check

server app4:8000 backup; # only when others down

keepalive 32; # connection pooling

}
```

```
server {
listen 80;
server_name api.myapp.com;

location /api/ {
proxy_pass http://backend;
proxy_http_version 1.1;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_connect_timeout 5s;
proxy_read_timeout 30s;
}

# Rate limiting
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
limit_req zone=api burst=20 nodelay;
}
```

INTERVIEW TIP

Layer 4 vs Layer 7 load balancing: L4 (TCP) routes based on IP/port — fast, no HTTP awareness. L7 (HTTP) routes based on URL path, headers, cookies — slower but enables path-based routing (/api -> backend, /static -> CDN). AWS ALB = L7, NLB = L4.

CHEAT SHEET ANCHOR

Load balancer: Round Robin(sequential), Least Conn(fewest active), Weighted(server capacity ratio), IP Hash(sticky). Nginx: upstream block with least_conn/ip_hash. Health check: server removed when failing. L4(TCP, fast) vs L7(HTTP, path-based routing). AWS ALB=L7, NLB=L4.

Q12 What is a RESTful API versioning strategy? Explain HATEOAS and API design patterns.

API VERSIONING STRATEGIES

```
# 1. URL Path Versioning (most common, most explicit)
GET /api/v1/users
GET /api/v2/users
# Pros: clear, easy to test in browser
```

```
# Cons: URL changes, clients must update

---

# 2. Query Parameter Versioning
GET /api/users?version=2
# Pros: single URL, easy to add
# Cons: query params should be for filtering not versioning

---

# 3. Header Versioning
GET /api/users
Accept: application/vnd.myapi.v2+json
# Pros: clean URLs
# Cons: harder to test, not visible in browser

---

# 4. Subdomain
GET https://v2.api.myapp.com/users
```

PAGINATION PATTERNS

```
# Offset Pagination (simple, for small datasets)
GET /users?page=3&limit=20
# offset = (page-1) * limit
# Problem: if records inserted during pagination, items appear twice

---

# Cursor Pagination (production standard)
GET /users?cursor=eyJ1c2VyX2lkIjoxMH0=&limit=20
# cursor = base64(last_item_id)
# WHERE id > last_seen_id ORDER BY id LIMIT 20
# Stable, no duplicates, works for infinite scroll

---

# Response with cursor
{
'data': [...],
'meta': {
'next_cursor': 'eyJ1c2VyX2lkIjozMH0=',
'has_more': true
}
}
```

API DESIGN BEST PRACTICES

- Use consistent error response format: {error: {code: 'USER_NOT_FOUND', message: '...', details: {...}}}.
- Idempotency keys for POST: X-Idempotency-Key header prevents duplicate payments on retry.
- Field selection: GET /users?fields=id,name,email — reduce over-fetching.
- Deprecation headers: Deprecation: true, Sunset: 2025-12-31 to warn before removing.
- Always validate input and return specific 400 errors with field-level details.

INTERVIEW TIP

Cursor pagination vs offset: cursor is the production standard for high-traffic APIs (Stripe, Twitter use it). Offset becomes incorrect when items are inserted/deleted mid-pagination (items can be skipped or doubled). Cursor is stable because it's based on a unique, immutable identifier.

CHEAT SHEET ANCHOR

API versioning: URL(/api/v1/, most common), Header(Accept: vnd.api+json), Query param(?version=2). Pagination: Offset(?page=3&limit=20, simple, unstable) vs Cursor(base64 encoded last ID, stable, production standard). Idempotency key for POST prevents duplicate processing.

Q13 What is WebSockets? When to use WebSockets vs SSE vs Long Polling?

REAL-TIME COMMUNICATION OPTIONS

Long Polling: client sends request, server holds it open until data available, then responds. Client immediately sends next request. Simple but inefficient.

Server-Sent Events (SSE): one-way server->client stream over HTTP. Simple, automatic reconnect. Good for: live feeds, notifications, dashboards.

WebSockets: full-duplex persistent connection. Both client and server can send messages anytime. Good for: chat, gaming, collaborative editing.

WEBSOCKET IMPLEMENTATION (Python FastAPI)

```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from typing import Dict, List

app = FastAPI()

class ConnectionManager:
def __init__(self):
self.active: Dict[str, List[WebSocket]] = {}
```

```
async def connect(self, ws: WebSocket, room: str):
await ws.accept()
self.active.setdefault(room, []).append(ws)

def disconnect(self, ws: WebSocket, room: str):
self.active.get(room, []).remove(ws)

async def broadcast(self, room: str, message: str):
for ws in self.active.get(room, []):
await ws.send_text(message)

manager = ConnectionManager()

@app.websocket('/ws/chat/{room_id}')
async def chat(ws: WebSocket, room_id: str):
await manager.connect(ws, room_id)
try:
while True:
data = await ws.receive_text()
await manager.broadcast(room_id, data)
except WebSocketDisconnect:
manager.disconnect(ws, room_id)
```

COMPARISON TABLE

```
Method | Direction | Protocol | Use Case
═══════════════════════════════════════════════════════════
Long Polling | S->C | HTTP | Legacy browsers, simple notifications
SSE | S->C only | HTTP | Live feeds, stock prices, notifications
WebSocket | Bi-directional | WS | Chat, games, collab editing
```

- SSE advantages over WebSockets: works through HTTP/2, automatic reconnection, standard HTTP headers (auth, CORS work natively).
- WebSocket advantages: bidirectional, lower overhead per message after handshake, better for high-frequency bidirectional data.

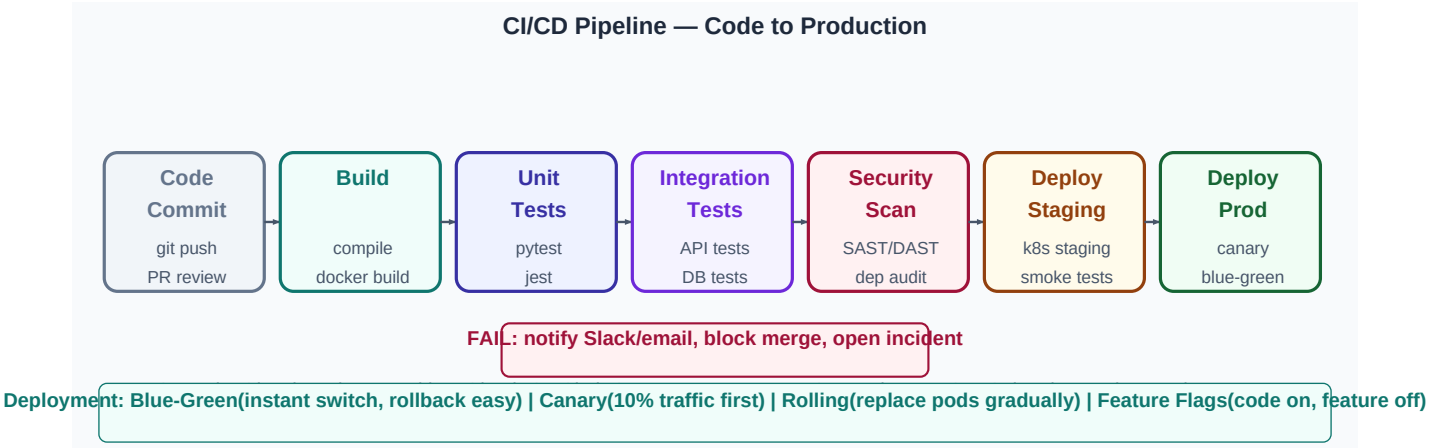
INTERVIEW TIP

SSE vs WebSocket: if you only need server-to-client updates (news feed, notifications, live scores), use SSE — it's simpler, works through standard HTTP proxies, and auto-reconnects. Use WebSockets only when you need true bidirectional communication (chat, multiplayer games, collaborative editors).

CHEAT SHEET ANCHOR

Long Polling=HTTP request held open. SSE=server-to-client only, HTTP, auto-reconnect, good for feeds. WebSocket=bidirectional persistent connection, WS protocol, good for chat/games. SSE simpler+HTTP native. WebSocket for bidirectional. Scale: Redis Pub/Sub to broadcast across multiple server instances.

Q14 What is a CI/CD pipeline? Explain stages and deployment strategies.



CI/CD: Code commit -> Build -> Unit Tests -> Integration Tests -> Security Scan -> Deploy Staging -> Deploy Production. Blue-Green/Canary for zero-downtime deploys.

CI/CD FUNDAMENTALS

CI (Continuous Integration): automatically build and test code on every commit. CD (Continuous Delivery/Deployment): automatically deploy to production after tests pass.

```
# GitHub Actions example (.github/workflows/deploy.yml)
name: Deploy to Production
on:
push:
branches: [main]
---
jobs:
test:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
```

```
- name: Set up Python
uses: actions/setup-python@v5
with: {python-version: '3.12'}
- run: pip install -r requirements.txt
- run: pytest --cov=app tests/
- run: ruff check . # linting
- run: bandit -r app/ # security scan
```

```
deploy:
needs: test
runs-on: ubuntu-latest
steps:
- name: Build Docker image
run: docker build -t myapp:$GITHUB_SHA .
- name: Push to registry
run: docker push myapp:$GITHUB_SHA
- name: Deploy to Kubernetes
run: kubectl set image deployment/myapp app=myapp:$GITHUB_SHA
```

DEPLOYMENT STRATEGIES

- Blue-Green: maintain two identical environments. Switch load balancer to new (green) instantly. Old (blue) kept for rollback. Zero downtime, easy rollback. Cost: 2x infrastructure.
- Canary: deploy to 5-10% of servers first. Monitor metrics/errors. Gradually increase to 100%. Best for risky changes.
- Rolling: replace pods/instances one at a time. Kubernetes default. Low cost, some old+new running simultaneously.
- Feature Flags: deploy code but control feature activation via config. Decouple deploy from release. Tools: LaunchDarkly, Unleash, custom Redis flag.

INTERVIEW TIP

The 'shift left' principle: catch bugs as early as possible. Linting catches style issues (cheapest). Unit tests catch logic bugs. Integration tests catch contract bugs. Security scans catch vulnerabilities. Each step is 10x more expensive to fix than the previous one if found in production.

CHEAT SHEET ANCHOR

CI=build+test on every commit. CD=auto-deploy after tests. GitHub Actions: push trigger->test job->deploy job. Deployment: Blue-Green(switch LB, zero downtime, 2x cost), Canary(10% first, monitor), Rolling(one-by-one, Kubernetes default), Feature Flags(code on, feature off).

Q15 What is middleware in web frameworks? Implement logging and error handling middleware.

WHAT IS MIDDLEWARE?

Middleware is code that runs before/after your request handlers. It forms a pipeline: each middleware can inspect/modify the request, call next(), then inspect/modify the response. Used for: logging, auth, rate limiting, CORS, error handling.

```
# Request -> Middleware1 -> Middleware2 -> Handler
# |
# Response <- Middleware1 <- Middleware2 <-----
```

MIDDLEWARE IN FASTAPI

```
import time, uuid, logging
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
from starlette.middleware.base import BaseHTTPMiddleware

app = FastAPI()

# Logging middleware
class LoggingMiddleware(BaseHTTPMiddleware):
async def dispatch(self, request: Request, call_next):
request_id = str(uuid.uuid4())
request.state.request_id = request_id
start = time.time()
logging.info(f'[{request_id}] {request.method} {request.url}')
response = await call_next(request)
duration = (time.time() - start) * 1000
logging.info(f'[{request_id}] {response.status_code} {duration:.1f}ms')
response.headers['X-Request-ID'] = request_id
response.headers['X-Response-Time'] = f'{duration:.1f}ms'
return response
```

```
# Global error handler
@app.exception_handler(Exception)
async def global_error_handler(request: Request, exc: Exception):
logging.error(f'Unhandled error: {exc}', exc_info=True)
return JSONResponse(
status_code=500,
content={
'error': {
'code': 'INTERNAL_ERROR',
'message': 'An unexpected error occurred',
'request_id': getattr(request.state, 'request_id', None)
}
}
)
```

```
app.add_middleware(LoggingMiddleware)
```

INTERVIEW TIP

Always include a unique request ID (X-Request-ID) in every request and log it in every log line. This makes distributed tracing possible — you can trace a single request across multiple services by searching for its ID in all logs. Generate if not provided by client.

CHEAT SHEET ANCHOR

Middleware: code runs before/after every request handler. Pipeline: request->MW1->MW2->handler->MW2->MW1->response. Uses: logging(X-Request-ID, timing), auth verification, rate limiting, CORS, error handling. Always log request_id for distributed tracing. Global exception handler returns 500 with context.

Q16 What is environment configuration management? How do you handle secrets and configs?

12-FACTOR APP CONFIGURATION

The 12-Factor App methodology says: store config in environment variables, never in code. Secrets (DB passwords, API keys) should never be committed to version control.

```
# config.py — read from environment variables
import os
from pydantic_settings import BaseSettings
```

```
class Settings(BaseSettings):
# Required — will raise error if missing
DATABASE_URL: str
SECRET_KEY: str
REDIS_URL: str

# Optional with defaults
DEBUG: bool = False
LOG_LEVEL: str = 'INFO'
WORKERS: int = 4
ALLOWED_ORIGINS: list = ['http://localhost:3000']

class Config:
env_file = '.env' # load from .env in development
env_file_encoding = 'utf-8'

settings = Settings()

# .env (local dev only, never commit this!)
DATABASE_URL=postgresql://user:password@localhost:5432/mydb
SECRET_KEY=your-256-bit-secret-key-here
REDIS_URL=redis://localhost:6379

# .gitignore
.env
.env.local
*.env
```

SECRETS MANAGEMENT IN PRODUCTION

- AWS Secrets Manager / Parameter Store: store secrets encrypted, rotate automatically. App fetches at startup or on demand.
- HashiCorp Vault: open-source, dynamic secrets (DB creds that expire after 1 hour), fine-grained access policies.
- Kubernetes Secrets: base64-encoded (not encrypted by default!). Use external-secrets-operator to sync from AWS/Vault.

```
# Fetch secret from AWS at startup
import boto3, json
```

```
def get_secret(secret_name):
client = boto3.client('secretsmanager', region_name='us-east-1')
response = client.get_secret_value(SecretId=secret_name)
return json.loads(response['SecretString'])

db_creds = get_secret('prod/myapp/database')
DB_URL = f'postgresql://{db_creds["user"]}:{db_creds["password"]}@...'
```

INTERVIEW TIP

The most common mistake: committing .env files or hardcoding API keys in code. Use git-secrets or pre-commit hooks to prevent secret commits. In production, never use .env files — use AWS Secrets Manager, Vault, or K8s secrets mounted as environment variables at runtime.

CHEAT SHEET ANCHOR

12-Factor App: config in environment variables, secrets never in code. Pydantic BaseSettings reads from env vars + .env file. .env in .gitignore always. Production: AWS Secrets Manager, HashiCorp Vault, K8s Secrets. Never commit credentials. Rotate secrets regularly.

Q17 What is database migration? Implement with Alembic and explain rollback strategies.

DATABASE MIGRATIONS

Database migrations are version-controlled schema changes. Like git for your database. Every schema change (add column, create table, create index) is a migration file that can be applied or reversed.

```
# Alembic setup with SQLAlchemy
# alembic.ini: sqlalchemy.url = %(DATABASE_URL)s

# env.py
from app.models import Base
target_metadata = Base.metadata

# Generate migration from model changes
# alembic revision --autogenerate -m 'add_phone_to_users'

# Generated migration file: 20240315_add_phone_to_users.py
def upgrade():
op.add_column('users',
sa.Column('phone', sa.String(20), nullable=True)
```

```
)

# Create index for the new column

op.create_index('ix_users_phone', 'users', ['phone'])

---

def downgrade():

op.drop_index('ix_users_phone', 'users')

op.drop_column('users', 'phone')

---

# Apply migrations

# alembic upgrade head # apply all pending

# alembic downgrade -1 # undo last migration

# alembic history # show migration history

# alembic current # show current version
```

ZERO-DOWNTIME MIGRATION STRATEGIES

- Expand-Contract pattern: Step 1 (Expand) — add new column as nullable. Step 2 — backfill data. Step 3 — add NOT NULL constraint. Step 4 (Contract) — remove old column.

✗ Never: DROP COLUMN or RENAME COLUMN on live table without coordination — running code reads the old column name.

- Always backward-compatible first: add columns as nullable, never remove columns until all code versions stop referencing them.

```
# Safe column rename (expand-contract)

# Step 1: add new column

op.add_column('users', sa.Column('full_name', sa.String(200)))

# Step 2: backfill (async, not in migration)

# Step 3: deploy code that writes to BOTH old and new

# Step 4: remove old column after all instances updated

op.drop_column('users', 'name')
```

INTERVIEW TIP

Never run migrations automatically on app startup in production — it's a common mistake. If two instances start simultaneously, both try to migrate and can corrupt the schema. Instead: run migrations as a separate pre-deployment step before starting new app instances.

CHEAT SHEET ANCHOR

Alembic: version-controlled schema changes. upgrade()+downgrade() functions. alembic upgrade head, downgrade -1. Zero-downtime: Expand(add nullable col)->Backfill->Contract(remove old). Never DROP/RENAME live columns without coordination. Run migrations as pre-deployment step, not at app startup.

Q18 What is API testing? Explain unit tests, integration tests, and contract testing.

TESTING PYRAMID

The testing pyramid: many cheap unit tests, fewer integration tests, fewest E2E tests. Each layer tests different things at different costs.

- Unit tests: test a single function in isolation. Fast, cheap, run on every commit. Mock all external dependencies.
- Integration tests: test multiple components together (API + DB, not mocked). Slower, fewer. Catch contract bugs.
- End-to-End tests: test full user flow in browser. Slowest, most brittle. Only for critical paths.

PYTEST API TESTING

```
import pytest
from fastapi.testclient import TestClient
from app.main import app
from app.database import get_db
```

```
# Unit test — mock database
def test_create_user_success(mocker):
mock_db = mocker.patch('app.routes.users.db')
mock_db.add_user.return_value = {'id': 1, 'name': 'Alice'}
result = create_user({'name': 'Alice', 'email': 'a@b.com'})
assert result['id'] == 1
mock_db.add_user.assert_called_once()
```

```
# Integration test — real test DB
@pytest.fixture
def client(test_db_session):
app.dependency_overrides[get_db] = lambda: test_db_session
return TestClient(app)
```

```
def test_create_user_api(client):
```

```
response = client.post('/api/users', json={
'name': 'Alice', 'email': 'alice@test.com', 'password': 'pass123'
})
assert response.status_code == 201
data = response.json()
assert data['email'] == 'alice@test.com'
assert 'password' not in data # never expose password
```

```
def test_duplicate_email(client):
client.post('/api/users', json={'email': 'dup@test.com'})
response = client.post('/api/users', json={'email': 'dup@test.com'})
assert response.status_code == 409 # Conflict
```

INTERVIEW TIP

Always test error cases: duplicate email, invalid input, unauthorized access, not found. Test that sensitive fields (password hashes, internal IDs) are not returned in responses. Use a separate test database that's reset between test runs — never test against production or shared dev DB.

CHEAT SHEET ANCHOR

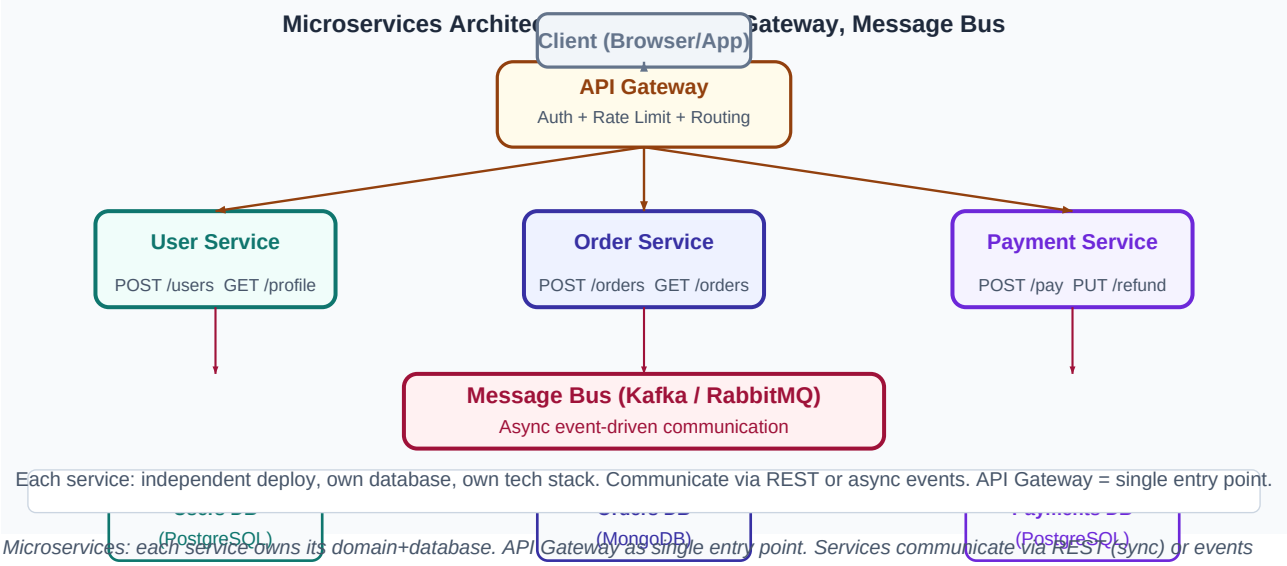
Testing pyramid: many unit(mock deps, fast) -> fewer integration(real DB, API) -> fewest E2E. pytest+TestClient for FastAPI. Key: test error cases(duplicate, unauthorized, not found). Never expose passwords in responses. Override dependency injection for test DB. Reset DB between tests.

LEVEL 2: INTERMEDIATE

Microservices, System Design, Message Queues, Design Patterns

Q19 - Q34

Q19 What is a microservices architecture? Explain service communication patterns and trade-offs.



Microservices: each service owns its domain+database. API Gateway as single entry point. Services communicate via REST (sync) or events (async). Message bus for decoupled async.

MICROSERVICES vs MONOLITH

```
Feature | Monolith | Microservices
────────────────────────────────────────────────────────────────────────────────
Deployment | Single unit | Independent services
Scale | Scale whole app | Scale per service
Dev team | One team | Multiple teams
Complexity | Simple at first | Distributed system
Data | Single DB | DB per service
Testing | Easy integration | Contract testing needed
Start with | Always start here | Only if team justifies
```

- Rule: start with a monolith, extract microservices when you have clear domain boundaries, independent scaling needs, or multiple teams stepping on each other.

SERVICE COMMUNICATION PATTERNS

Synchronous (REST/gRPC): caller waits for response. Use for: queries that need immediate answer, simple request-response.

Asynchronous (Events/Messages): fire and forget, caller doesn't wait. Use for: long-running tasks, cross-service side effects.

```
# Service discovery — how services find each other
# Client-side: Eureka/Consul — service registers itself
# Server-side: Kubernetes Service — DNS-based (svc-name:port)
```

```
# Circuit breaker — fail fast when downstream is down
from circuitbreaker import circuit
```

```
@circuit(failure_threshold=5, recovery_timeout=30)
def call_payment_service(order_id):
response = requests.post(f'{PAYMENT_URL}/pay', ...)
return response.json()
# After 5 failures: circuit OPENS, fail immediately for 30s
# No cascading failures: one service down doesn't kill all
```

API GATEWAY RESPONSIBILITIES

- Authentication: verify JWT/session before forwarding to services.
- Rate limiting: throttle by user/IP before requests hit services.
- Request routing: /api/users/* -> User Service, /api/orders/* -> Order Service.
- Request aggregation: combine responses from multiple services into one (BFF pattern).
- SSL termination: handle HTTPS at gateway, services use plain HTTP internally.

INTERVIEW TIP

Circuit breaker is the most important resilience pattern in microservices. Without it, when Payment Service is slow (30s timeout), all Order Service threads get stuck waiting, Order Service becomes slow, and the whole system cascades into failure. Circuit breaker breaks the chain by failing fast.

CHEAT SHEET ANCHOR

Microservices: independent services, own DB, independent deploy. API Gateway=single entry(auth+rate limit+routing). Sync=REST/gRPC(immediate response). Async=events(Kafka, decouple). Circuit breaker: 5 failures->open circuit->fail fast 30s->prevent cascade. Start monolith, extract when needed.

Q20 What is the Repository Pattern? Implement it with dependency injection for testability.

REPOSITORY PATTERN

The Repository Pattern abstracts the data layer behind an interface. Business logic depends on the interface, not the concrete database implementation. Makes testing easy (swap real DB with in-memory fake).

```
from abc import ABC, abstractmethod

from typing import Optional, List

from dataclasses import dataclass

---

@dataclass

class User:

id: int

name: str

email: str

---

# Interface

class UserRepository(ABC):

@abstractmethod

def get_by_id(self, user_id: int) -> Optional[User]: ...

@abstractmethod

def get_by_email(self, email: str) -> Optional[User]: ...

@abstractmethod

def create(self, name: str, email: str) -> User: ...

@abstractmethod

def update(self, user: User) -> User: ...

@abstractmethod

def delete(self, user_id: int) -> None: ...

---

# PostgreSQL implementation

class PostgresUserRepository(UserRepository):

def __init__(self, db_session):

self.session = db_session

---

def get_by_id(self, user_id: int) -> Optional[User]:

row = self.session.execute(

'SELECT * FROM users WHERE id=%s', (user_id,)

).fetchone()
```

```
return User(**row) if row else None

def create(self, name: str, email: str) -> User:
row = self.session.execute(
'INSERT INTO users(name,email) VALUES(%s,%s) RETURNING *',
(name, email)
).fetchone()
self.session.commit()
return User(**row)
```

DEPENDENCY INJECTION + FASTAPI

```
from fastapi import Depends

# In-memory repository for tests
class InMemoryUserRepository(UserRepository):
def __init__(self):
self._users: dict = {}
self._next_id = 1

def get_by_id(self, user_id: int) -> Optional[User]:
return self._users.get(user_id)

def create(self, name: str, email: str) -> User:
user = User(id=self._next_id, name=name, email=email)
self._users[self._next_id] = user
self._next_id += 1
return user

# Service layer — depends on interface, not implementation
class UserService:
def __init__(self, repo: UserRepository):
self.repo = repo

def register(self, name: str, email: str) -> User:
existing = self.repo.get_by_email(email)
if existing:
raise ValueError(f'Email {email} already registered')
```

```
return self.repo.create(name, email)

# FastAPI dependency injection
def get_user_repo(db=Depends(get_db)) -> UserRepository:
return PostgresUserRepository(db)

@router.post('/users')
def create_user(data: CreateUserRequest,
repo: UserRepository = Depends(get_user_repo)):
service = UserService(repo)
return service.register(data.name, data.email)

# Test: swap with in-memory repo
def test_register():
repo = InMemoryUserRepository()
service = UserService(repo)
user = service.register('Alice', 'alice@test.com')
assert user.email == 'alice@test.com'
```

INTERVIEW TIP

Dependency injection is the key to testable code. When your service receives its dependencies via constructor (not creates them internally), you can swap real implementations for fakes in tests. This is why 'don't new up dependencies in your class' is a fundamental clean code principle.

CHEAT SHEET ANCHOR

Repository Pattern: abstract interface + concrete implementations(Postgres, InMemory). Business logic depends on interface not DB. DI: inject repo into service via constructor. FastAPI Depends() for DI. Test: swap with InMemoryRepository, no real DB needed. Testable by design.

Q21 What is the Saga pattern? Implement orchestration-based saga for distributed transactions.

THE SAGA PATTERN

Saga manages distributed transactions across multiple microservices. Each service executes a local transaction and publishes an event. If any step fails, compensating transactions undo previous steps.

```
# Order Placement Saga
# Steps: CreateOrder -> ReserveInventory -> ChargePayment -> ConfirmOrder
# Compensation: CancelOrder <- ReleaseInventory <- RefundPayment
```

ORCHESTRATION-BASED SAGA

```
from enum import Enum
from dataclasses import dataclass, field
from typing import List, Callable
```

```
class SagaStep:
def __init__(self, name, action, compensation):
self.name = name
self.action = action # forward action
self.compensation = compensation # rollback action
```

```
class SagaOrchestrator:
def __init__(self, steps: List[SagaStep]):
self.steps = steps
```

```
def execute(self, context: dict):
completed = []
for step in self.steps:
try:
print(f'Executing step: {step.name}')
step.action(context)
completed.append(step)
except Exception as e:
print(f'Step {step.name} failed: {e}')
# Compensate in reverse order
for done_step in reversed(completed):
try:
print(f'Compensating: {done_step.name}')
done_step.compensation(context)
except Exception as ce:
print(f'Compensation failed: {ce}')
# Log to DLQ for manual intervention
raise
```

```
# Usage
```

```
def create_order_saga(order_data):
context = {'order': order_data}
saga = SagaOrchestrator([
SagaStep('CreateOrder',
action=lambda ctx: order_svc.create(ctx),
compensation=lambda ctx: order_svc.cancel(ctx)),
SagaStep('ReserveInventory',
action=lambda ctx: inventory_svc.reserve(ctx),
compensation=lambda ctx: inventory_svc.release(ctx)),
SagaStep('ChargePayment',
action=lambda ctx: payment_svc.charge(ctx),
compensation=lambda ctx: payment_svc.refund(ctx)),
])
saga.execute(context)
```

INTERVIEW TIP

The most critical saga implementation detail: what happens when compensation itself fails? You need a Dead Letter Queue + alerting + manual intervention process for stuck sagas. Log every saga step + its outcome to a saga log table so you can always see what state a saga is in.

CHEAT SHEET ANCHOR

Saga: distributed transaction via local txns + compensating txns. Orchestration: central coordinator calls each service. If step N fails: compensate steps N-1..1 in reverse. Choreography: event-driven, no coordinator. Compensation can fail -> DLQ + alerting. Log saga state for debugging.

Q22 What is the Outbox Pattern? Implement reliable event publishing from a database transaction.

THE DUAL-WRITE PROBLEM

Naive approach: save to DB, then publish to Kafka. Problem: DB succeeds but Kafka fails -> order saved but event never published -> inventory never decremented.

```
# WRONG: dual-write — non-atomic
def create_order(order):
db.save(order) # Write 1
kafka.publish(OrderCreated(order)) # Write 2 — can fail!
# If kafka fails: order exists but no event published
```

OUTBOX PATTERN IMPLEMENTATION

```
# Schema: outbox table in same database
CREATE TABLE outbox (
id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
event_type VARCHAR(100) NOT NULL,
payload JSONB NOT NULL,
status VARCHAR(20) DEFAULT 'PENDING',
created_at TIMESTAMPTZ DEFAULT NOW(),
published_at TIMESTAMPTZ
);
```

```
# Save order + outbox entry in ONE transaction
def create_order_with_outbox(order_data, db):
with db.transaction():
order = Order(**order_data)
db.add(order)
db.flush() # get order.id
```

```
outbox_entry = OutboxEvent(
event_type='OrderCreated',
payload={
'order_id': order.id,
'user_id': order.user_id,
'amount': float(order.amount),
'items': order.items,
'timestamp': order.created_at.isoformat()
}
)
db.add(outbox_entry)
# Both committed atomically or both rolled back
```

```
# Background relay: publish outbox -> Kafka
import asyncio
async def outbox_relay(db, kafka_producer):
while True:
```

```
pending = db.query(OutboxEvent)\
.filter_by(status='PENDING')\
.order_by(OutboxEvent.created_at)\
.limit(100)\
.with_for_update(skip_locked=True) # avoid double processing
.all()
for event in pending:
kafka_producer.send(event.event_type, event.payload)
event.status = 'PUBLISHED'
event.published_at = datetime.utcnow()
db.commit()
await asyncio.sleep(0.5)
```

INTERVIEW TIP

FOR UPDATE SKIP LOCKED is the critical PostgreSQL feature for the outbox relay. It allows multiple relay instances to run in parallel without processing the same event twice — each instance locks the rows it picks up and skips already-locked rows. This is PostgreSQL's built-in advisory locking for queue processing.

CHEAT SHEET ANCHOR

Outbox: save business data + event in SAME database transaction -> atomicity guaranteed. Background relay reads outbox+publishes to Kafka+marks published. SKIP LOCKED prevents duplicate processing by multiple relay instances. Debezium via CDC can replace polling relay(reads WAL directly).

Q23 What is the CQRS pattern? Implement separate read and write models.

CQRS — COMMAND QUERY RESPONSIBILITY SEGREGATION

CQRS separates write operations (commands: create, update, delete) from read operations (queries: search, list, get). Each side can be optimized independently — write side for consistency, read side for performance.

```
# Write model: normalized PostgreSQL for consistency
class OrderCommandHandler:
def __init__(self, order_repo, event_publisher):
self.repo = order_repo
self.publisher = event_publisher
---
def handle_create_order(self, cmd: CreateOrderCommand):
```

```
# Validate
if cmd.amount <= 0:
raise ValueError('Amount must be positive')
# Write to normalized DB
order = self.repo.create(cmd.user_id, cmd.items, cmd.amount)
# Publish event for read model update
self.publisher.publish('OrderCreated', {
'order_id': order.id,
'user_id': cmd.user_id,
'amount': cmd.amount,
'items': cmd.items,
'created_at': order.created_at.isoformat()
})
return order.id
```

READ MODEL — ELASTICSEARCH / REDIS

```
from elasticsearch import Elasticsearch
```

```
# Read model: denormalized for fast queries
class OrderReadModel:
def __init__(self):
self.es = Elasticsearch()
```

```
def get_order_history(self, user_id, filters=None):
query = {
'query': {
'bool': {
'must': [{'term': {'user_id': user_id}}],
'filter': []
}
},
'sort': [{'created_at': 'desc'}]
}
return self.es.search(index='orders', body=query)
```

```
# Event handler: update read model from events
@kafka_consumer('OrderCreated')
def on_order_created(event):
# Enrich with user details (denormalize)
user = user_service.get_user(event['user_id'])
read_model.es.index(index='orders', id=event['order_id'],
body={
**event,
'user_name': user['name'],
'user_email': user['email'],
})
```

INTERVIEW TIP

CQRS's main benefit: the read model can be completely optimized for queries without affecting write consistency. A complex dashboard query that would require 5 JOINs in normalized PostgreSQL becomes a single document read from Elasticsearch. The tradeoff: eventual consistency between write and read models.

CHEAT SHEET ANCHOR

CQRS: write side(commands, normalized DB, consistency) + read side(queries, denormalized cache/ES, performance). Event handler syncs read model from write events. Eventual consistency tradeoff. Benefits: independent scaling+optimization. Use when read/write perf needs differ significantly.

Q24 What is gRPC? How does it differ from REST and when should you use it?

gRPC OVERVIEW

gRPC is a high-performance RPC framework using Protocol Buffers (binary serialization) over HTTP/2. Supports streaming, strongly typed contracts via .proto files, and auto-generated client/server code.

```
# 1. Define service in .proto file
# user_service.proto
syntax = 'proto3';
package users;
```

```
message User {
int32 id = 1;
string name = 2;
```

```
string email = 3;
}

message GetUserRequest { int32 id = 1; }
message CreateUserRequest {
string name = 1;
string email = 2;
}

service UserService {
rpc GetUser(GetUserRequest) returns (User);
rpc CreateUser(CreateUserRequest) returns (User);
rpc StreamUsers(GetUserRequest) returns (stream User);
}

# 2. Generate Python code
# python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. user_service.proto

# 3. Server implementation
import grpc
import user_service_pb2, user_service_pb2_grpc

class UserServiceImpl(user_service_pb2_grpc.UserServiceServicer):
def GetUser(self, request, context):
user = db.get_user(request.id)
if not user:
context.set_code(grpc.StatusCode.NOT_FOUND)
context.set_details(f'User {request.id} not found')
return user_service_pb2.User()
return user_service_pb2.User(
id=user.id, name=user.name, email=user.email)

def StreamUsers(self, request, context):
for user in db.get_all_users():
yield user_service_pb2.User(id=user.id, name=user.name)
```

```
Feature | REST | gRPC | GraphQL
────────────────────────────────────────────────────────────────────────────────
Protocol | HTTP/1.1 | HTTP/2 | HTTP/1.1 or 2
Format | JSON (text) | ProtoBuf (binary) | JSON (text)
Schema | OpenAPI | .proto files | SDL
Streaming | SSE/WS | Native (4 types) | Subscriptions
Perf | Good | Excellent | Good
Browser | Native | Not supported | Native
Best for | Public API | Internal services | Complex frontends
```

INTERVIEW TIP

gRPC vs REST: REST for public APIs (browser accessible, human readable JSON). gRPC for internal microservice-to-microservice communication (binary, faster, streaming, strong typing). You can use both: gRPC between services, REST for public-facing API gateway.

CHEAT SHEET ANCHOR

gRPC: Protocol Buffers binary over HTTP/2. Define in .proto file, auto-generate client+server code. 4 streaming types: unary, server stream, client stream, bidirectional. 5-10x faster than REST for internal services. Not browser-native. Use: internal microservices. REST: public APIs.

Q25 What is observability? Implement logging, metrics, and distributed tracing.

THE THREE PILLARS OF OBSERVABILITY

- Logs: timestamped text records of events. What happened? Best for debugging specific errors.
- Metrics: numerical measurements over time (request rate, latency, error rate, CPU). Best for monitoring trends and alerting.
- Traces: end-to-end request journey across multiple services. Best for latency diagnosis in distributed systems.

STRUCTURED LOGGING

```
import structlog, json
────────────────────────────────────────────────────────────────────────────────
# Configure structured logging (JSON output)
structlog.configure(
processors=[
structlog.processors.TimeStamper(fmt='iso'),
structlog.processors.StackInfoRenderer(),
```

```
structlog.processors.JSONRenderer(),
]
)
log = structlog.get_logger()
```

```
# Good: structured, searchable
log.info('order_created',
order_id=12345,
user_id=99,
amount=149.99,
duration_ms=42)
# Output: {'event':'order_created','order_id':12345,'timestamp':'...'}
```

```
# Bad: unstructured, unsearchable
print(f'Order 12345 for user 99 created, amount 149.99')
```

METRICS AND DISTRIBUTED TRACING

```
from prometheus_client import Counter, Histogram, start_http_server
```

```
REQUEST_COUNT = Counter('http_requests_total',
'Total HTTP requests',
['method', 'endpoint', 'status'])
REQUEST_LATENCY = Histogram('http_request_duration_seconds',
'HTTP request latency',
buckets=[.005,.01,.025,.05,.1,.25,.5,1,2.5])
```

```
@app.middleware('http')
async def metrics_middleware(request, call_next):
start = time.time()
response = await call_next(request)
REQUEST_LATENCY.observe(time.time() - start)
REQUEST_COUNT.labels(
method=request.method,
endpoint=request.url.path,
status=response.status_code),
).inc()
```

```
return response
```

```
# OpenTelemetry distributed tracing
from opentelemetry import trace
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
FastAPIInstrumentor.instrument_app(app)
# Automatic trace context propagation across services
# View in Jaeger/Zipkin/Honeycomb
```

INTERVIEW TIP

The SRE golden signals: Latency (how long requests take), Traffic (how many requests), Errors (rate of failed requests), Saturation (how full the system is). Alert on these four. P99 latency (99th percentile) is more important than average — average hides outliers that real users experience.

CHEAT SHEET ANCHOR

Observability 3 pillars: Logs(structured JSON, what happened), Metrics(numbers over time, Prometheus, alert on SLOs), Traces(request journey across services, OpenTelemetry, Jaeger). Golden signals: Latency+Traffic+Errors+Saturation. P99 latency more important than average.

Q26 What is a service mesh? Explain Istio and sidecar proxy pattern.

SERVICE MESH

A service mesh manages service-to-service communication in a microservices architecture via sidecar proxies (Envoy) injected alongside each service. Handles: mutual TLS, retries, circuit breaking, load balancing, observability — without changing application code.

```
# Every pod gets a sidecar proxy injected by Kubernetes
Pod structure:
+------ Pod ------+
| App Container | <- your service
| Envoy Proxy | <- injected sidecar
+-----------------+
```

```
# All traffic flows through sidecar:
# Service A (Envoy) --> Service B (Envoy)
# mTLS automatically applied between sidecars
```

ISTIO CONFIGURATION

```
# Istio VirtualService: traffic routing
```

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
name: user-service
spec:
hosts: [user-service]
http:
- match:
- headers:
x-version:
exact: 'v2'
route:
- destination:
host: user-service
subset: v2
weight: 100
- route:
- destination:
host: user-service
subset: v1
weight: 90
- destination:
host: user-service
subset: v2
weight: 10 # 10% canary
```

```
# Circuit breaker via DestinationRule
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
spec:
trafficPolicy:
outlierDetection:
consecutive5xxErrors: 5
```

```
interval: 30s
baseEjectionTime: 60s
# 5 errors -> eject from pool for 60s
```

INTERVIEW TIP

Service mesh solves the 'cross-cutting concerns' problem — retry logic, mTLS, circuit breaking, observability would need to be implemented in every service's code. With a mesh, it's infrastructure-level. The tradeoff: operational complexity of running Istio. For smaller deployments, implement these patterns in code instead.

CHEAT SHEET ANCHOR

Service mesh: sidecar proxy(Envoy) injected into every pod, handles mTLS+retries+circuit breaking+observability without code changes. Istio: control plane(Istiod) + data plane(Envoy sidecars). VirtualService=traffic routing(canary, header-based). DestinationRule=circuit breaking. Tradeoff: ops complexity.

Q27 What is database connection pooling? Configure PgBouncer for production PostgreSQL.

THE PROBLEM WITHOUT POOLING

Creating a PostgreSQL connection requires: TCP handshake, authentication, session setup. Costs 20-100ms and ~10MB RAM. A web app handling 1,000 req/s creating a new connection per request would overwhelm PostgreSQL's max_connections (typically 100-300).

PGBOUNCER CONFIGURATION

```
# pgbouncer.ini
[databases]
mydb = host=postgres-primary port=5432 dbname=mydb
---
[pgbouncer]
listen_port = 6432
listen_addr = 0.0.0.0
auth_type = md5
auth_file = /etc/pgbouncer/userlist.txt
---
# Pool mode (most important setting)
pool_mode = transaction # release after each txn
# Options: session (release on disconnect)
# transaction (release after txn — recommended)
```

```
# statement (release after each statement)

# Connection limits
max_client_conn = 10000 # clients to PgBouncer
default_pool_size = 20 # connections to PostgreSQL
min_pool_size = 5
reserve_pool_size = 5 # emergency connections

# Timeouts
pool_timeout = 5 # wait for connection
server_idle_timeout = 600 # close idle server conns
client_idle_timeout = 0 # disabled

# SQLAlchemy connects to PgBouncer, not PostgreSQL directly
DATABASE_URL = 'postgresql://user:pass@pgbouncer:6432/mydb'

# With SQLAlchemy in transaction mode:
engine = create_engine(DATABASE_URL,
pool_pre_ping=True, # check connection alive
pool_size=5, # SQLAlchemy -> PgBouncer pool
max_overflow=10,
pool_recycle=300)
```

POOL SIZE FORMULA

```
# PgBouncer pool_size per pool:
# Optimal = min(max_connections * 0.9, 2 * db_cpus + spindles)
# For 8-core Postgres: pool_size = 2*8 = 16 connections
# With PgBouncer: 10,000 clients -> 20 actual DB connections
```

- Transaction mode: connection released after each transaction. Most efficient. Cannot use: session-level features (SET, prepared statements in some configs, LISTEN/NOTIFY).
- Session mode: connection held for entire client session. Less efficient but safe for all features.

INTERVIEW TIP

Pool size of 20 connections can handle thousands of requests per second. More is not better — too many connections cause context switching and actually slow down PostgreSQL. The PgBouncer documentation's recommended formula is 2*num_cpu_cores for the actual PostgreSQL connections.

CHEAT SHEET ANCHOR

Connection pooling: reuse pre-established connections(avoid 50-100ms setup each time). PgBouncer: 10,000 clients->20 actual DB connections. pool_mode=transaction(release after txn, most efficient). pool_size=2*num_cpus recommended. SQLAlchemy still pools to PgBouncer. pool_pre_ping=True for stale detection.

Q28 What is the strangler fig pattern? How do you migrate from monolith to microservices?

STRANGLER FIG PATTERN

The Strangler Fig pattern (Martin Fowler) migrates a monolith to microservices incrementally. A facade/proxy sits in front of both. New features go to microservices. Old features are extracted one by one. The monolith is 'strangled' gradually, never big-bang rewritten.

```
# Migration steps:
# 1. Put proxy/API gateway in front of monolith
# 2. Identify bounded context to extract (e.g., User Service)
# 3. Build new microservice alongside monolith
# 4. Route specific traffic to new service (feature flag or URL)
# 5. Migrate data if needed (dual-write period)
# 6. Remove old code from monolith
# 7. Repeat for next bounded context
```

TRAFFIC ROUTING DURING MIGRATION

```
# Nginx routing: new /api/users -> new User Service
location /api/users {
proxy_pass http://user-microservice;
}
# Everything else still goes to monolith
location / {
proxy_pass http://monolith;
}
```

```
# Data migration: dual-write period
# Both monolith and new service write to both DBs temporarily
def create_user(user_data):
# Write to old monolith DB
monolith_db.save(user_data)
```

```
# Write to new service DB
user_service_db.save(user_data)
# After cutover: remove monolith write
```

```
# Feature flag for gradual rollout
def get_user(user_id):
if feature_flag.is_enabled('use_user_microservice', user_id):
return requests.get(f'{USER_SERVICE_URL}/users/{user_id}')
return monolith_db.get_user(user_id)
```

WHAT TO EXTRACT FIRST

- Start with: bounded contexts with clear ownership, high change rate, or independent scaling needs.
- Avoid extracting: core domain logic with many integrations — too many seams to cut.
- Good first candidates: auth/user service, notification service, file upload service — relatively independent, stable interfaces.

INTERVIEW TIP

The most dangerous mistake in monolith-to-microservices migration: big bang rewrite. This almost always fails (the second system effect). The strangler fig approach is safer because: you never have a day when nothing works, you can validate each extracted service before removing the monolith code, and you can stop at any point.

CHEAT SHEET ANCHOR

Strangler Fig: proxy in front of monolith, extract one bounded context at a time, route traffic to new service, dual-write during data migration, remove monolith code. Never big-bang rewrite. Start with: independent services(notification, file upload, auth). Feature flags for gradual traffic shift.

Q29 What is API Gateway pattern? Implement one using Python with authentication and routing.

API GATEWAY RESPONSIBILITIES

- Authentication: verify JWT/API key before forwarding. Services don't need to handle auth.
- Rate limiting: throttle abusive clients before they hit services.
- Request routing: forward requests to correct service based on path/header.
- Request/Response transformation: add headers, strip sensitive fields.
- Aggregation (BFF): combine responses from multiple services for a single client request.
- SSL termination: handle HTTPS externally, services use plain HTTP internally.

SIMPLE API GATEWAY IN PYTHON

```
import httpx, jwt
from fastapi import FastAPI, Request, HTTPException
from fastapi.responses import Response

app = FastAPI()
SECRET = 'your-secret'

# Service registry
ROUTES = {
'/api/users': 'http://user-service:8001',
'/api/orders': 'http://order-service:8002',
'/api/payments': 'http://payment-service:8003',
}

def get_service_url(path: str) -> str:
for prefix, url in ROUTES.items():
if path.startswith(prefix):
return url + path
raise HTTPException(404, 'Route not found')

def verify_token(token: str) -> dict:
try:
return jwt.decode(token, SECRET, algorithms=['HS256'])
except jwt.InvalidTokenError:
raise HTTPException(401, 'Invalid token')

@app.api_route('/{path:path}', methods=['GET','POST','PUT','DELETE','PATCH'])
async def proxy(request: Request, path: str):
# Skip auth for login endpoint
if not path.startswith('api/auth/'):
auth = request.headers.get('Authorization', '')
token = auth.split(' ')[-1] if auth.startswith('Bearer ') else ''
user_payload = verify_token(token)

service_url = get_service_url('/' + path)
```

```
body = await request.body()

async with httpx.AsyncClient() as client:
response = await client.request(
method=request.method,
url=service_url,
headers={
**dict(request.headers),
'X-User-ID': str(user_payload.get('sub', '')),
},
content=body,
timeout=30.0
)
return Response(content=response.content,
status_code=response.status_code,
headers=dict(response.headers))
```

INTERVIEW TIP

Production API gateways (Kong, AWS API Gateway, Traefik) provide authentication, rate limiting, logging, and routing out of the box. Building your own is educational but not recommended for production unless you have very specific requirements. Start with an existing battle-tested solution.

CHEAT SHEET ANCHOR

API Gateway: single entry point, handles auth+rate limit+routing+aggregation+SSL termination. Dowstreams use X-User-ID header instead of verifying JWT. Pattern: proxy with auth middleware. Production: Kong, AWS API Gateway, Traefik. Custom only for specific needs.

Q30 What is Kubernetes? Explain pods, services, deployments, and ingress.

KUBERNETES CORE CONCEPTS

```
# Pod: smallest deployable unit, 1+ containers
# Deployment: manages replicas of pods, rolling updates
# Service: stable DNS name + load balancer for pods
# Ingress: HTTP routing from outside cluster to services

# deployment.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: api-server
spec:
replicas: 3
selector:
matchLabels:
app: api-server
strategy:
type: RollingUpdate
rollingUpdate:
maxSurge: 1
maxUnavailable: 0 # zero downtime
template:
metadata:
labels:
app: api-server
spec:
containers:
- name: api
image: myapp:v1.2.3
ports: [{containerPort: 8000}]
env:
- name: DATABASE_URL
valueFrom:
secretKeyRef:
name: app-secrets
key: DATABASE_URL
resources:
requests: {cpu: '100m', memory: '128Mi'}
limits: {cpu: '500m', memory: '512Mi'}
livenessProbe:
```

```
httpGet: {path: /health, port: 8000}
initialDelaySeconds: 30
readinessProbe:
httpGet: {path: /ready, port: 8000}
initialDelaySeconds: 5
```

SERVICE AND INGRESS

```
# service.yaml — stable DNS name for pods
apiVersion: v1
kind: Service
metadata: {name: api-server}
spec:
selector: {app: api-server}
ports: [{port: 80, targetPort: 8000}]
type: ClusterIP # internal only
```

```
# ingress.yaml — external HTTP routing
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
name: api-ingress
annotations:
nginx.ingress.kubernetes.io/rate-limit: '100'
spec:
tls: [{hosts: [api.myapp.com], secretName: tls-secret}]
rules:
- host: api.myapp.com
http:
paths:
- path: /api/users
pathType: Prefix
backend:
service: {name: user-svc, port: {number: 80}}
```

INTERVIEW TIP

Liveness vs Readiness probes: liveness=is the container alive (restart if fails). Readiness=is the container ready to serve traffic (remove from load balancer if fails, but don't restart). A container can be alive but not ready (warming up cache, DB migration running). Always implement both.

CHEAT SHEET ANCHOR

K8s: Pod(1+ containers), Deployment(manages pods, rolling update), Service(stable DNS+LB for pods), Ingress(external HTTP routing). resources.requests=guaranteed, limits=max. Liveness probe=restart if dead. Readiness probe=remove from LB if not ready. Secrets via secretKeyRef.

Q31 What is database indexing in production? Explain partial, composite, and covering indexes.

PRODUCTION INDEXING STRATEGIES

```
-- Always explain before indexing

EXPLAIN ANALYZE

SELECT * FROM orders WHERE user_id = 42 AND status = 'pending';

-- If you see: Seq Scan on orders (rows=10M) -> needs index

---

-- Basic index

CREATE INDEX CONCURRENTLY idx_orders_user_id ON orders(user_id);

-- CONCURRENTLY: builds without locking table (production safe)

---

-- Composite index (column order matters!)

CREATE INDEX idx_orders_user_status ON orders(user_id, status);

-- Efficient for: WHERE user_id=42 AND status='pending'

-- Efficient for: WHERE user_id=42 (leftmost prefix)

-- NOT efficient for: WHERE status='pending' alone

---

-- Partial index (index only relevant subset)

CREATE INDEX idx_orders_pending ON orders(created_at)

WHERE status = 'pending';

-- Smaller index: only pending orders

-- Perfect for: WHERE status='pending' ORDER BY created_at

---

-- Covering index (include extra columns to avoid table lookup)

CREATE INDEX idx_orders_covering

ON orders(user_id, status)
```

```
INCLUDE (amount, created_at);

-- Query SELECT amount, created_at WHERE user_id=42 AND status='pending'

-- fetches ALL data from index, never touches table (index-only scan)

-- Check index usage

SELECT indexname, idx_scan, idx_tup_read

FROM pg_stat_user_indexes

WHERE relname = 'orders'

ORDER BY idx_scan DESC;
```

INDEX MAINTENANCE

- Bloated indexes: over time, B-Tree indexes accumulate dead tuples. REINDEX CONCURRENTLY rebuilds without locking.
- Unused indexes: every index slows down INSERT/UPDATE/DELETE. Drop indexes with idx_scan=0 after 30+ days.
- pg_stat_statements: find slow queries that need indexing.

```
-- Find missing indexes via slow query log

SELECT query, mean_exec_time, calls

FROM pg_stat_statements

ORDER BY mean_exec_time DESC LIMIT 10;
```

INTERVIEW TIP

CONCURRENTLY is critical in production. CREATE INDEX (without CONCURRENTLY) takes a table lock — no inserts/updates during build. CONCURRENTLY allows normal writes but takes longer. Always use CONCURRENTLY for production tables unless the table is empty or you have a maintenance window.

CHEAT SHEET ANCHOR

Index: CREATE INDEX CONCURRENTLY(no table lock, production safe). Composite: (user_id, status) - leftmost prefix rule. Partial: WHERE status='pending'(smaller, faster). Covering: INCLUDE(amount, created_at) - avoids table lookup. Check: pg_stat_user_indexes(unused indexes slow writes). EXPLAIN ANALYZE first.

Q32 What is background job processing? Implement with Celery and Redis.

BACKGROUND JOBS — WHY

Long-running tasks (email sending, PDF generation, image processing) should not block HTTP request-response cycle. Offload to background workers, return task ID to client, client polls for completion.

```
# Without background jobs:
POST /send-report # takes 30 seconds, client times out

# With background jobs:
POST /send-report # returns {task_id: 'xyz'} in <100ms
GET /tasks/xyz # poll: {status: 'running'} or {status: 'done'}
```

CELERY + REDIS IMPLEMENTATION

```
from celery import Celery
from celery.utils.log import get_task_logger

app = Celery('tasks',
broker='redis://localhost:6379/0',
backend='redis://localhost:6379/1',
include=['tasks'])

app.conf.update(
task_serializer = 'json',
result_expires = 3600,
task_max_retries = 3,
worker_prefetch_multiplier = 1, # fair distribution
)

log = get_task_logger(__name__)

@app.task(bind=True, max_retries=3)
def send_email(self, to: str, subject: str, body: str):
try:
log.info(f'Sending email to {to}')
email_service.send(to, subject, body)
return {'status': 'sent', 'to': to}
except Exception as exc:
log.error(f'Email failed: {exc}')
raise self.retry(exc=exc, countdown=60) # retry in 60s

@app.task
def generate_report(report_id: int):
```

```
data = db.get_report_data(report_id)
pdf_bytes = pdf_generator.create(data)
s3.upload(f'reports/{report_id}.pdf', pdf_bytes)
db.update_report_status(report_id, 'done')
return {'url': f's3://bucket/reports/{report_id}.pdf'}
```

```
# FastAPI endpoint
@router.post('/reports/{report_id}/generate')
async def trigger_report(report_id: int):
task = generate_report.delay(report_id)
return {'task_id': task.id, 'status': 'queued'}
```

```
@router.get('/tasks/{task_id}')
async def get_task_status(task_id: str):
task = generate_report.AsyncResult(task_id)
return {'status': task.status, 'result': task.result}
```

```
# Start worker: celery -A tasks worker --loglevel=info -c 4
# Monitor: flower (celery flower --port=5555)
```

INTERVIEW TIP

Celery retry with exponential backoff: retry(exc=exc, countdown=2**self.request.retries * 60) — first retry in 60s, second in 120s, third in 240s. This is the production pattern for transient failures (network blip, rate limit). Always set max_retries to prevent infinite retry loops.

CHEAT SHEET ANCHOR

Background jobs: HTTP returns task_id immediately, worker processes async. Celery+Redis: @task decorator, .delay() to enqueue, .AsyncResult() to check status. Retry with exponential backoff. Separate queues per priority(emails vs reports). Monitor with Flower. Dead tasks: manual inspect + retry from admin.

Q33 What is GraphQL? Implement a GraphQL API with queries, mutations, and N+1 fix.

GRAPHQL BASICS

GraphQL is a query language for APIs. Client specifies exactly what data it needs. Single endpoint. Solves over-fetching (getting too many fields) and under-fetching (needing multiple requests).

```
# Schema definition
import strawberry
```

```
from typing import List, Optional
```

```
@strawberry.type
class User:
id: int
name: str
email: str
posts: List['Post']
```

```
@strawberry.type
class Post:
id: int
title: str
content: str
author: User
```

```
@strawberry.type
class Query:
@strawberry.field
def user(self, id: int) -> Optional[User]:
row = db.fetchone('SELECT * FROM users WHERE id=%s', id)
return User(**row) if row else None
```

```
@strawberry.field
def users(self) -> List[User]:
return [User(**row) for row in db.fetchall('SELECT * FROM users')]
```

```
@strawberry.type
class Mutation:
@strawberry.mutation
def create_user(self, name: str, email: str) -> User:
row = db.fetchone(
'INSERT INTO users(name,email) VALUES(%s,%s) RETURNING *',
name, email)
return User(**row)
```

N+1 FIX IN GRAPHQL — DATALOADER

```
from strawberry.dataloader import DataLoader

# Without DataLoader: N+1 (1 query per user to get posts)
# With DataLoader: batch all user_ids into 1 IN query

async def load_posts_for_users(user_ids: List[int]) -> List[List[Post]]:
# ONE query for all user IDs
rows = db.fetchall(
'SELECT * FROM posts WHERE user_id = ANY(%s)', user_ids)
posts_by_user = defaultdict(list)
for row in rows:
posts_by_user[row['user_id']].append(Post(**row))
return [posts_by_user[uid] for uid in user_ids]

posts_loader = DataLoader(load_fn=load_posts_for_users)

@strawberry.type
class User:
id: int
@strawberry.field
async def posts(self) -> List[Post]:
return await posts_loader.load(self.id) # batched!

# Query: {users { name posts { title } }}
# Without DataLoader: 1 + N queries
# With DataLoader: 2 total queries
```

INTERVIEW TIP

DataLoader is essential for any GraphQL implementation — without it, every nested field causes N+1 queries. The DataLoader collects all load(id) calls within one tick of the event loop and batches them into a single IN query. This is the GraphQL equivalent of SQL's JOIN.

CHEAT SHEET ANCHOR

GraphQL: single endpoint, client specifies exact fields needed, no over/under-fetching. Strawberry=Python GraphQL. Queries=reads, Mutations=writes, Subscriptions=realtime. N+1 fix: DataLoader batches all IDs in one tick -> single IN query. schema-first or code-first both valid.

Q34 What is OAuth 2.0? Explain the authorization code flow with PKCE.

OAUTH 2.0 FLOWS

OAuth 2.0 is an authorization framework for third-party access delegation. 'Login with Google' uses OAuth. There are 4 grant types for different scenarios.

- Authorization Code + PKCE: web/mobile apps. Most secure. Standard for all new apps.
- Client Credentials: server-to-server (machine-to-machine). No user involved.
- Implicit: deprecated (insecure, tokens in URL). Do not use.
- Resource Owner Password: deprecated (anti-pattern). Do not use.

AUTHORIZATION CODE FLOW WITH PKCE

```
import secrets, hashlib, base64, requests

# Step 1: Client generates PKCE verifier and challenge
code_verifier = secrets.token_urlsafe(64)
code_challenge = base64.urlsafe_b64encode(
hashlib.sha256(code_verifier.encode()).digest()
).rstrip(b'=').decode()

# Step 2: Redirect user to authorization server
state = secrets.token_urlsafe(32) # CSRF protection
auth_url = (
f'https://accounts.google.com/o/oauth2/v2/auth'
f'?response_type=code'
f'&client_id={CLIENT_ID}'
f'&redirect_uri=https://myapp.com/callback'
f'&scope=openid%20email%20profile'
f'&code_challenge={code_challenge}'
f'&code_challenge_method=S256'
f'&state={state}'
)
# Store (state, code_verifier) in session

# Step 3: Handle callback
def oauth_callback(code, state):
```

```
# Verify state matches session (CSRF check)
assert state == session['oauth_state']
```

```
# Exchange code for tokens
token_response = requests.post('https://oauth2.googleapis.com/token', {
'grant_type': 'authorization_code',
'code': code,
'redirect_uri': 'https://myapp.com/callback',
'client_id': CLIENT_ID,
'client_secret': CLIENT_SECRET,
'code_verifier': session['code_verifier'], # PKCE
})
tokens = token_response.json()
# tokens: {access_token, refresh_token, id_token, expires_in}
```

```
# Decode ID token (user info without extra request)
user_info = jwt.decode(tokens['id_token'], options={'verify_signature': False})
# {sub: '107691503500061507151130823 67', email: 'user@gmail.com', ...}
```

INTERVIEW TIP

PKCE (Proof Key for Code Exchange) prevents authorization code interception. Without PKCE, if someone intercepts the auth code, they can exchange it for tokens. With PKCE, they also need the code_verifier (which was only sent in the first request and never appears in the URL). Use PKCE for all OAuth flows, not just mobile apps.

CHEAT SHEET ANCHOR

OAuth 2.0: authorization framework for delegated access. Auth Code+PKCE: most secure, generate verifier+challenge(SHA256), redirect to auth server, callback with code, exchange code+verifier for tokens. State parameter=CSRF protection. PKCE prevents code interception. Never use Implicit or ROPC flows.

LEVEL 3: ADVANCED

Distributed Systems, Consistency, Consensus, Scaling

Q35 - Q50

Q35 What is the SOLID principle applied to backend architecture? Give concrete Python examples.

SOLID PRINCIPLES IN PRACTICE

S — Single Responsibility: a class/function does ONE thing and has ONE reason to change.

```
# BAD: UserService does too much
class UserService:
def register(self, user): ...
def send_welcome_email(self, user): ...
def generate_pdf_report(self, user): ...
def update_analytics(self, user): ...
```

```
# GOOD: separate responsibilities
class UserService:
def register(self, user): ... # only user logic
class EmailService:
def send_welcome(self, user): ... # only email
```

O — Open/Closed: open for extension, closed for modification.

```
# BAD: modify class to add new payment
class PaymentProcessor:
def process(self, method):
if method == 'stripe': ...
elif method == 'paypal': ... # keep modifying
```

```
# GOOD: extend via new class
class PaymentProvider(ABC):
@abstractmethod
def charge(self, amount): ...
```

```
class StripeProvider(PaymentProvider):
def charge(self, amount): ...
```

```
class PayPalProvider(PaymentProvider):
def charge(self, amount): ...
```

L — Liskov Substitution: subclasses must be substitutable for base class.

I — Interface Segregation: don't force classes to implement unused methods.

```
# BAD: fat interface
class Repository(ABC):
def get(self): ...
def create(self): ...
def update(self): ...
def bulk_import(self): ... # not all repos need this
```

```
# GOOD: split interfaces
class ReadRepository(ABC):
def get(self): ...
class WriteRepository(ReadRepository):
def create(self): ...
def update(self): ...
```

D — Dependency Inversion: high-level modules depend on abstractions, not implementations.

```
# BAD: UserService creates its own DB
class UserService:
def __init__(self):
self.db = PostgresDB() # tightly coupled
```

```
# GOOD: inject dependency
class UserService:
def __init__(self, repo: UserRepository): # abstraction
self.repo = repo
```

INTERVIEW TIP

SOLID is not about following rules rigidly — it's about managing change. SRP makes code easier to change. OCP prevents bugs from modifications. DIP enables testing. In practice, the most impactful are SRP (keep things small and focused) and DIP (inject dependencies, don't create them inside classes).

CHEAT SHEET ANCHOR

SOLID: S=one class one responsibility. O=extend via new class not modify existing. L=subclass substitutable for parent. I=fat interfaces->split into small specific ones. D=depend on abstractions not implementations(inject UserRepository not PostgresDB). DIP enables testability.

Q36 What is event-driven architecture? Implement an event sourcing system.

EVENT SOURCING OVERVIEW

Event sourcing stores the HISTORY of events that led to current state, instead of just current state. Each event is immutable and append-only. Current state is rebuilt by replaying events.

```
# Traditional: store current balance
accounts: id=1, balance=4500 # history lost
```

```
# Event sourcing: store all events
events:
AccountOpened: account_id=1, amount=10000
MoneyWithdrawn: account_id=1, amount=2000
MoneyDeposited: account_id=1, amount=1000
MoneyWithdrawn: account_id=1, amount=4500
# Current state = replay: 10000 - 2000 + 1000 - 4500 = 4500
```

EVENT SOURCING IMPLEMENTATION

```
from dataclasses import dataclass, field
from typing import List, Dict, Any
from datetime import datetime
import json
```

```
@dataclass
class Event:
event_type: str
payload: Dict[str, Any]
version: int
occurred_at: datetime = field(default_factory=datetime.utcnow)
```

```
class BankAccount:
def __init__(self, account_id: int):
self.account_id = account_id
```

```
self.balance = 0.0
self._events: List[Event] = []
self._version = 0
```

```
def open(self, initial_balance: float):
if initial_balance <= 0:
raise ValueError('Initial balance must be positive')
self._apply(Event('AccountOpened',
{'amount': initial_balance}, self._version+1))
```

```
def withdraw(self, amount: float):
if amount > self.balance:
raise ValueError('Insufficient balance')
self._apply(Event('MoneyWithdrawn',
{'amount': amount}, self._version+1))
```

```
def deposit(self, amount: float):
self._apply(Event('MoneyDeposited',
{'amount': amount}, self._version+1))
```

```
def _apply(self, event: Event):
# Apply event to update state
if event.event_type == 'AccountOpened':
self.balance = event.payload['amount']
elif event.event_type == 'MoneyWithdrawn':
self.balance -= event.payload['amount']
elif event.event_type == 'MoneyDeposited':
self.balance += event.payload['amount']
self._version = event.version
self._events.append(event)
```

```
@classmethod
def rebuild_from_events(cls, account_id, events):
account = cls(account_id)
for event in events:
account._apply(event)
```

```
return account

# Persistence: save events, not state
def save_account(account: BankAccount, event_store):
for event in account._events:
event_store.append(account.account_id, event)

# Load: rebuild from events
def load_account(account_id: int, event_store) -> BankAccount:
events = event_store.load(account_id)
return BankAccount.rebuild_from_events(account_id, events)
```

INTERVIEW TIP

Snapshots are essential for event sourcing at scale. If an account has 10 million events, replaying all of them on every load is slow. Snapshot: periodically save the current state + version number. On load: fetch latest snapshot, then replay only events since snapshot version.

CHEAT SHEET ANCHOR

Event sourcing: store events not state, current state=replay all events. Events are immutable+append-only. Benefits: audit trail, time travel, debugging. Snapshots: save state+version periodically to avoid replaying 10M events. Combined with CQRS: events build read models.

Q37 What is a distributed cache? Explain Redis Cluster, consistency, and cache patterns.

REDIS CLUSTER ARCHITECTURE

Redis Cluster shards data across multiple Redis nodes using consistent hashing over 16,384 hash slots. No single point of failure with master-replica pairs per shard.

```
# Redis Cluster: 6 nodes (3 masters + 3 replicas)
# Master 1: slots 0-5460 -> Replica 1
# Master 2: slots 5461-10922 -> Replica 2
# Master 3: slots 10923-16383-> Replica 3

# Key placement: HASH_SLOT = CRC16(key) % 16384
# 'user:123' always goes to same slot -> same master

import redis.cluster
cluster = redis.cluster.RedisCluster(
```

```
startup_nodes=[
{'host': 'redis-1', 'port': 6379},
{'host': 'redis-2', 'port': 6379},
],
decode_responses=True,
skip_full_coverage_check=True
)
cluster.set('user:123', json.dumps(user_data), ex=3600)
```

ADVANCED CACHE PATTERNS

Cache warming: pre-populate cache on startup to avoid cold start thundering herd.

```
async def warm_cache():
top_users = db.fetchall('SELECT * FROM users ORDER BY activity DESC LIMIT 1000')
pipeline = redis.pipeline()
for user in top_users:
pipeline.setex(f'user:{user.id}', 3600, json.dumps(user))
await pipeline.execute()
```

Cache stampede prevention (probabilistic early expiration):

```
def get_with_jitter(key, fetch_fn, ttl=3600):
data = cache.get(key)
if data is None:
with redis_lock(key + ':lock', timeout=5):
data = cache.get(key) # double-check
if data is None:
data = fetch_fn()
jitter = random.randint(-60, 60) # spread expiry
cache.setex(key, ttl + jitter, data)
return data
```

Read-through cache: cache acts as intermediary, auto-fetches from DB on miss.

Write-around: writes go to DB only, cache populated on subsequent reads.

INTERVIEW TIP

Cache stampede (thundering herd): when a popular key expires, thousands of simultaneous requests all miss the cache and hammer the database. Fix: probabilistic early expiration (refresh before expiry) + mutex lock (only one request rebuilds, others wait). Jitter on TTL prevents synchronized expirations.

CHEAT SHEET ANCHOR

Redis Cluster: 16384 hash slots across 3+ master nodes with replicas. HASH_SLOT=CRC16(key)%16384. Multi-key ops require hash tags {user}:123. Cache stampede fix: mutex lock + double-check + TTL jitter. Cache warming: pre-populate on startup. Pipeline for batch operations.

Q38 What is the Twelve-Factor App methodology? Apply it to a production backend.

12-FACTOR APP PRINCIPLES

- 1. Codebase: one codebase tracked in VCS, many deploys (dev/staging/prod).
- 2. Dependencies: explicitly declare and isolate (requirements.txt, pyproject.toml).
- 3. Config: store config in environment variables, not in code.
- 4. Backing services: treat DB, Redis, S3 as attached resources (swap via URL).
- 5. Build/Release/Run: strict separation. Build=code+deps. Release=build+config. Run=execute.
- 6. Processes: stateless processes, share nothing. No local filesystem state.
- 7. Port binding: export services via port. App is self-contained web server.
- 8. Concurrency: scale out via processes (multiple Gunicorn workers, multiple containers).
- 9. Disposability: fast startup, graceful shutdown. Handle SIGTERM correctly.
- 10. Dev/prod parity: keep dev, staging, prod as similar as possible.
- 11. Logs: treat as event streams, write to stdout, aggregate centrally.
- 12. Admin processes: run one-off tasks as process in same environment (flask db upgrade).

GRACEFUL SHUTDOWN (FACTOR 9)

```
import signal, asyncio

from fastapi import FastAPI

app = FastAPI()

shutdown_event = asyncio.Event()

@app.on_event('startup')

async def startup():

# Initialize DB pools, Redis connections, etc.
```

```
await db_pool.connect()
await redis_pool.connect()
```

```
@app.on_event('shutdown')
async def shutdown():
# Allow in-flight requests to complete
await asyncio.sleep(5) # grace period
await db_pool.close()
await redis_pool.close()
```

```
# Kubernetes sends SIGTERM before killing pod
# Gunicorn config: graceful_timeout = 30
# workers = 2 * cpu_count + 1 (for sync workers)
# worker_class = 'uvicorn.workers.UvicornWorker' (async)
```

STATELESS PROCESSES (FACTOR 6)

- Never store user state in process memory — if the load balancer sends next request to different instance, state is lost.
- Store session state in Redis (not in-memory).
- Store uploaded files in S3 (not local filesystem).
- Store task state in database/Redis (not Celery worker memory).

INTERVIEW TIP

Factor 6 (stateless processes) is the most violated in practice. The symptom: 'It works on my server but not on others' — because state is stored in memory on one server. Fix: always store state externally (Redis, DB, S3). This enables horizontal scaling (just add more instances).

CHEAT SHEET ANCHOR

12-Factor App: 1.One codebase, 3.Config in env vars, 6.Stateless processes(no in-memory state), 9.Graceful shutdown(SIGTERM->drain->close), 11.Logs to stdout. Factor 6=scale horizontally by adding instances. Session in Redis not memory. Files in S3 not local disk.

Q39 What is database sharding in a backend application? Implement horizontal sharding.

APPLICATION-LEVEL SHARDING

Sharding splits data across multiple database instances by a shard key. The application routes each request to the correct shard. Used when a single PostgreSQL instance cannot handle the data volume or write throughput.

```
import hashlib
from sqlalchemy import create_engine
from contextlib import contextmanager

class ShardManager:
def __init__(self, shard_configs: list):
self.shards = [
create_engine(cfg['url'],
pool_size=10, max_overflow=5)
for cfg in shard_configs
]
self.num_shards = len(self.shards)

def get_shard(self, shard_key: str):
# Consistent hash: shard_key -> shard index
hash_value = int(hashlib.md5(
str(shard_key).encode()).hexdigest(), 16)
shard_index = hash_value % self.num_shards
return self.shards[shard_index]

@contextmanager
def session(self, shard_key: str):
engine = self.get_shard(shard_key)
with engine.connect() as conn:
yield conn

# Usage
shard_mgr = ShardManager([
{'url': 'postgresql://...shard1'},
{'url': 'postgresql://...shard2'},
{'url': 'postgresql://...shard3'},
])
```

```
def get_user(user_id: int) -> dict:
# user_id as shard key: same user always on same shard
with shard_mgr.session(str(user_id)) as db:
return db.execute(
'SELECT * FROM users WHERE id=%s', user_id).fetchone()
```

```
def create_order(user_id: int, order_data: dict):
# Co-locate user's orders on same shard as user
with shard_mgr.session(str(user_id)) as db:
return db.execute(
'INSERT INTO orders(user_id, ...) VALUES(%s, ...) RETURNING *',
user_id, ...).fetchone()
```

SHARDING CHALLENGES IN APPLICATIONS

✗ Cross-shard queries: SELECT * FROM users WHERE email='x' — must query all shards and merge.

✗ Cross-shard transactions: order (shard1) + payment (shard2) — use Saga pattern, not 2PC.

✗ Re-sharding: adding a 4th shard requires moving ~25% of data — complex, needs double-write period.

• Data co-location: store user, orders, payments on same shard (user_id as shard key for all).

• Consistent hashing: adding servers moves only K/N keys (not full rehash).

INTERVIEW TIP

Always exhaust these options before sharding: vertical scaling (bigger server), read replicas (scale reads), caching (Redis), query optimization, table partitioning. Sharding is a last resort that massively increases operational complexity. Many companies with millions of users never need to shard.

CHEAT SHEET ANCHOR

Application sharding: route requests to correct DB shard by hash(shard_key)%num_shards. Co-locate related data(user+orders on same shard). Challenges: cross-shard queries(query all, merge), cross-shard txns(Saga not 2PC), re-sharding(double-write migration). Consistent hashing for smooth scaling.

Q40 What are database transactions and isolation levels in a web application?

TRANSACTION ISOLATION IN PRACTICE

```
# Flask-SQLAlchemy transaction example
from sqlalchemy import text
```

```
def transfer_funds(from_id, to_id, amount):
```

```
with db.session.begin(): # auto-commit or rollback
# Lock both accounts to prevent concurrent modification
from_acc = db.session.execute(
text('SELECT * FROM accounts WHERE id=:id FOR UPDATE'),
{'id': from_id}
).fetchone()
if from_acc.balance < amount:
raise ValueError('Insufficient balance')
db.session.execute(
text('UPDATE accounts SET balance=balance-:amt WHERE id=:id'),
{'amt': amount, 'id': from_id}
)
db.session.execute(
text('UPDATE accounts SET balance=balance+:amt WHERE id=:id'),
{'amt': amount, 'id': to_id}
)
# Both committed atomically or both rolled back
```

ISOLATION LEVELS AND WHEN TO USE

```
# Set isolation level in SQLAlchemy
from sqlalchemy import create_engine
# READ COMMITTED (default in PostgreSQL)
engine = create_engine(url,
execution_options={'isolation_level': 'READ COMMITTED'})
# REPEATABLE READ — for consistent snapshots
with db.session.begin():
db.session.execute(text(
'SET TRANSACTION ISOLATION LEVEL REPEATABLE READ'))
count1 = db.session.execute(text('SELECT COUNT(*) FROM orders')).scalar()
# ... some processing ...
count2 = db.session.execute(text('SELECT COUNT(*) FROM orders')).scalar()
# count1 == count2 guaranteed in REPEATABLE READ
```

```
# SERIALIZABLE — for critical financial operations
with db.session.begin():
db.session.execute(text('SET TRANSACTION ISOLATION LEVEL SERIALIZABLE'))
# Prevents: lost updates, phantom reads, write skew
```

- PostgreSQL default READ COMMITTED: fine for most web app operations.
- REPEATABLE READ: use for complex reports that read same data multiple times.
- SERIALIZABLE: use for critical financial operations. Much slower.

INTERVIEW TIP

Write skew is a subtle bug that only SERIALIZABLE prevents: Two transactions both read a condition as true, and based on that, both update — but neither update alone would have violated the condition. Example: hospital scheduling where two doctors both see they're the only doctor on call and both try to take the day off.

CHEAT SHEET ANCHOR

Transaction isolation: READ COMMITTED(default PG, ok for most). REPEATABLE READ(same data consistent in one txn). SERIALIZABLE(all anomalies prevented, slow). FOR UPDATE=row lock prevents concurrent modification. Deadlock: lock in consistent order. Use SERIALIZABLE for financial write skew prevention.

Q41 What is a content delivery network (CDN)? How does it integrate with a backend?

CDN ARCHITECTURE

A CDN (Content Delivery Network) caches content at edge servers geographically close to users, reducing latency and origin server load. Static files (JS, CSS, images) served from CDN. API responses can also be cached at the edge.

- Edge servers: data centers in major cities worldwide (100+ locations for CloudFront, Cloudflare).
- Origin: your backend servers. CDN fetches from origin on cache miss.
- TTL: how long CDN caches content before re-fetching from origin.

CDN CACHE-CONTROL HEADERS

```
# Backend sets cache headers to control CDN behavior
```

```
# Static assets — cache 1 year at CDN and browser
@app.route('/static/<path:filename>')
def static_files(filename):
```

```
response = send_file(f'static/{filename}')
response.headers['Cache-Control'] = 'public, max-age=31536000, immutable'
# immutable: browser won't revalidate even if page refreshes
return response
```

```
# API response — cache at CDN only (not browser), 60 seconds
@app.route('/api/products')
def list_products():
products = get_all_products()
response = jsonify(products)
response.headers['Cache-Control'] = 's-maxage=60, max-age=0'
# s-maxage: CDN cache duration
# max-age=0: browser must revalidate (always fresh for user)
response.headers['Surrogate-Key'] = 'products' # Cloudflare tag
return response
```

```
# Cache invalidation on update
def update_product(product_id, data):
db.update_product(product_id, data)
# Purge CDN cache for this product
cloudflare.purge_by_tag('products') # Cloudflare Cache Tags
# or
cloudflare.purge_urls([
f'https://myapp.com/api/products/{product_id}',
'https://myapp.com/api/products',
])
```

EDGE FUNCTIONS — LOGIC AT CDN

• Cloudflare Workers / AWS Lambda@Edge: run JavaScript at CDN edge. Use for: A/B testing at edge, geo-based routing, bot protection, authentication without round-trip to origin.

```
# Cloudflare Worker: auth check at edge
addEventListener('fetch', event => {
event.respondWith(handleRequest(event.request))
})
async function handleRequest(request) {
```

```
const token = request.headers.get('Authorization')
if (!isValid(token)) {
return new Response('Unauthorized', {status: 401})
}
return fetch(request) // forward to origin
}
```

INTERVIEW TIP

Cache-Control: s-maxage is CDN-specific TTL (overrides max-age for CDN). max-age is browser TTL. 'immutable' tells browsers that the URL content will never change (safe for content-addressed URLs like main.a1b2c3d4.js). Always use content-addressed filenames for static assets (webpack handles this).

CHEAT SHEET ANCHOR

CDN: edge servers cache static+dynamic content near users. Cache-Control: public,max-age(browser+CDN), s-maxage(CDN only), immutable(never revalidate). Cache invalidation: purge by URL or cache tags(Cloudflare). Edge functions: run code at CDN(auth, geo-routing, A/B). Origin=your backend.

Q42 What is a search engine integration? Implement Elasticsearch in a backend application.

ELASTICSEARCH BASICS

Elasticsearch is a distributed search and analytics engine. Stores data as JSON documents in indices. Uses inverted index for full-text search. Best for: product search, log analytics, full-text, autocomplete.

```
from elasticsearch import Elasticsearch, helpers
```

```
es = Elasticsearch(['http://localhost:9200'])
```

```
# Create index with mappings
es.indices.create(index='products', body={
'mappings': {
'properties': {
'name': {'type': 'text', 'analyzer': 'english'},
'description':{'type': 'text', 'analyzer': 'english'},
'category': {'type': 'keyword'}, # exact match
'price': {'type': 'float'},
'in_stock': {'type': 'boolean'},
```

```
'created_at': {'type': 'date'},
}
}
}, ignore=400)
```

```
# Index a document
es.index(index='products', id=product.id, body={
'name': product.name,
'description': product.description,
'category': product.category,
'price': product.price,
'in_stock': product.stock > 0,
})
```

SEARCH QUERIES

```
def search_products(query, category=None, min_price=None,
max_price=None, page=1, size=20):
must = [{'multi_match': {
'query': query,
'fields': ['name^3', 'description'], # name 3x boost
'type': 'best_fields',
'fuzziness': 'AUTO' # typo tolerance
}}]
filters = [{'term': {'in_stock': True}}]
if category:
filters.append({'term': {'category': category}})
if min_price or max_price:
filters.append({'range': {'price': {
'gte': min_price, 'lte': max_price}}})
```

```
response = es.search(index='products', body={
'query': {'bool': {'must': must, 'filter': filters}},
'sort': [{'_score': 'desc'}, {'price': 'asc'}],
'from': (page-1)*size,
'size': size,
```

```
'highlight': {'fields': {'name': {}, 'description': {}}}
})
return {
'total': response['hits']['total']['value'],
'results': [hit['_source'] for hit in response['hits']['hits']],
}
```

KEEPING ES IN SYNC WITH POSTGRESQL

- Dual-write: write to DB then ES in same request (simple but ES can fail).
- Debezium CDC: stream PostgreSQL WAL changes to Kafka, Kafka connector writes to ES. Eventually consistent, no data loss.
- Background sync: DB is source of truth, background job syncs to ES every N seconds. Simpler but has lag.

INTERVIEW TIP

Elasticsearch is eventually consistent with PostgreSQL — there's always a slight lag between DB write and ES availability. Never use ES as the source of truth for financial or critical data. Only use ES for search/discovery. Lookups by exact ID (product page) should still go to PostgreSQL.

CHEAT SHEET ANCHOR

Elasticsearch: inverted index, JSON documents, distributed. text(full-text+analyzer) vs keyword(exact match). Query: multi_match(multi-field), bool(must+filter), range, fuzziness(typo tolerance). Score=relevance. Boost fields with ^3. Sync via Debezium CDC or dual-write. ES=search only, PG=source of truth.

Q43 What is an idempotency key? Implement idempotent API endpoints for payment processing.

THE IDEMPOTENCY PROBLEM

Network failures cause uncertainty: did my POST /pay request reach the server? If client retries, payment might be charged twice. Idempotency keys solve this: if client sends same key twice, server guarantees same result (not double-charged).

```
# Without idempotency:
POST /payments {amount: 100} -> 201 Created, charge $100
# Network drops, client doesn't receive response
POST /payments {amount: 100} -> 201 Created, charge $100 AGAIN
# Customer charged $200 for one order!
```

IDEMPOTENCY KEY IMPLEMENTATION

```
import hashlib, json
from fastapi import Header, HTTPException

async def process_payment_idempotent(
payment_data: PaymentRequest,
idempotency_key: str = Header(None, alias='X-Idempotency-Key')
):
if not idempotency_key:
raise HTTPException(400, 'X-Idempotency-Key header required')

# Check if we've seen this key before
cache_key = f'idempotency:{idempotency_key}'
cached = await redis.get(cache_key)

if cached:
# Return same response as before (not re-process)
return json.loads(cached)

# Acquire distributed lock to prevent concurrent processing
lock_key = f'idempotency_lock:{idempotency_key}'
async with redis_lock(lock_key, timeout=30):
# Double-check (another request might have completed)
cached = await redis.get(cache_key)
if cached:
return json.loads(cached)

# Process payment
result = await stripe.charge(
amount=payment_data.amount,
currency='usd',
payment_method=payment_data.payment_method_id
)
response = {
'payment_id': result.id,
'status': result.status,
'amount': payment_data.amount
```

```
}

# Cache response for 24 hours (survive client retries)

await redis.setex(cache_key, 86400, json.dumps(response))

return response
```

INTERVIEW TIP

The idempotency key must be client-generated (UUID v4) and unique per logical operation. If the client doesn't know the result (network drop), it retries with the SAME key. The server returns the cached result without re-processing. Cache the response for at least 24 hours (longer than any reasonable retry window).

CHEAT SHEET ANCHOR

Idempotency key: client-generated UUID per logical operation. Server: check Redis cache first, if exists return cached response, else process+cache. Redis lock prevents concurrent processing of same key. TTL=24h(longer than retry window). Required for: payments, order creation, anything non-idempotent.

Q44 What is feature flags? Implement a feature flag system for gradual rollout.

FEATURE FLAGS OVERVIEW

Feature flags (feature toggles) decouple code deployment from feature release. Deploy code to production with the feature disabled. Enable it for 1% of users, monitor, then gradually increase to 100%.

- Use cases: canary releases, A/B testing, kill switches, beta programs, trunk-based development.

FEATURE FLAG IMPLEMENTATION

```
import redis, hashlib

from functools import lru_cache

class FeatureFlag:

def __init__(self, redis_client):

self.redis = redis_client

def is_enabled(self, flag_name: str, user_id: int = None) -> bool:

# Get flag config from Redis

config = self.redis.hgetall(f'feature_flag:{flag_name}')

if not config:

return False # unknown flag = disabled

enabled = config.get('enabled', '0') == '1'

if not enabled:
```

```
return False

# Percentage rollout
rollout_pct = int(config.get('rollout_pct', '100'))
if rollout_pct < 100 and user_id is not None:
# Deterministic: same user always in/out
bucket = int(hashlib.md5(
f'{flag_name}:{user_id}'.encode()
).hexdigest(), 16) % 100
return bucket < rollout_pct

return True

def set_flag(self, flag_name: str, enabled: bool,
rollout_pct: int = 100):
self.redis.hset(f'feature_flag:{flag_name}', mapping={
'enabled': '1' if enabled else '0',
'rollout_pct': str(rollout_pct)
})

# Usage in API
flags = FeatureFlag(redis_client)

@router.get('/checkout')
async def checkout(user: User = Depends(get_current_user)):
if flags.is_enabled('new_checkout_flow', user.id):
return new_checkout_handler(user)
return legacy_checkout_handler(user)

# Enable for 10% of users
flags.set_flag('new_checkout_flow', enabled=True, rollout_pct=10)
```

INTERVIEW TIP

Deterministic bucketing is the key: hash(flag_name + user_id) % 100 < rollout_pct. This ensures the same user always gets the same experience (not randomly switching on each request). It also means you can gradually increase rollout_pct and only NEW users join the flag, existing users stay in their bucket.

CHEAT SHEET ANCHOR

Feature flags: decouple deployment from release. is_enabled(flag, user_id): check Redis config+rollout_pct. Deterministic: hash(flag+user_id)%100<pct ensures same user always in/out. Set flag via admin API. Use: canary rollout, A/B test, kill switch, beta program. Tools: LaunchDarkly, Unleash, or custom Redis.

Q45 What is API security? Implement OWASP Top 10 protections in a backend API.

OWASP TOP 10 FOR APIS

A01 Broken Access Control: user can access other users' resources.

```
# BAD: only checks auth, not ownership
@app.get('/orders/{order_id}')
def get_order(order_id: int, user=Depends(get_current_user)):
return db.get_order(order_id) # anyone can access any order!
```

```
# GOOD: verify ownership
@app.get('/orders/{order_id}')
def get_order(order_id: int, user=Depends(get_current_user)):
order = db.get_order(order_id)
if order.user_id != user.id and 'admin' not in user.roles:
raise HTTPException(403, 'Access denied')
return order
```

A03 Injection: SQL injection, command injection.

```
# BAD: string concatenation
db.execute(f'SELECT * FROM users WHERE name = {user_input}')
# GOOD: parameterized query
db.execute('SELECT * FROM users WHERE name = %s', (user_input,))
```

A04 Insecure Design: mass assignment vulnerability.

```
# BAD: directly assign all request fields
user.update(**request.json()) # user can set is_admin=True!
```

```
# GOOD: whitelist allowed fields
ALLOWED_FIELDS = {'name', 'email', 'bio'}
safe_data = {k: v for k, v in request.json().items()
if k in ALLOWED_FIELDS}
user.update(**safe_data)
```

A07 Authentication failures: weak passwords, no brute force protection.

```
# Brute force protection
def login(email, password):
fail_key = f'login_failures:{email}'
failures = int(redis.get(fail_key) or 0)
if failures >= 5:
raise HTTPException(429, 'Too many failed attempts')
user = verify_credentials(email, password)
if not user:
redis.incr(fail_key)
redis.expire(fail_key, 300) # 5 min window
raise HTTPException(401, 'Invalid credentials')
redis.delete(fail_key) # reset on success
return generate_token(user)
```

INTERVIEW TIP

Broken Object Level Authorization (BOLA) / IDOR is the #1 API security bug. The pattern: GET /orders/{id} returns data for any id — not just the logged-in user's orders. Every endpoint that takes an ID must verify the current user owns that resource. This should be tested in every integration test.

CHEAT SHEET ANCHOR

OWASP API Security: A01 Broken Access Control(IDOR-verify ownership), A03 Injection(parameterized queries always), A04 Mass Assignment(whitelist fields), A07 Auth failures(rate limit login, bcrypt passwords). Every endpoint: verify auth+ownership. Never trust user input.

Q46 What is a reverse proxy? Implement request routing and SSL termination with Nginx.

REVERSE PROXY FUNCTIONS

- SSL termination: handle HTTPS at proxy, backend uses plain HTTP.
- Load balancing: distribute requests across multiple backend instances.
- Caching: cache static and dynamic responses, reduce origin load.
- Compression: gzip responses before sending to client.
- Security: hide backend topology, WAF (Web Application Firewall) integration.
- Request/response modification: add security headers, strip internal headers.

PRODUCTION NGINX CONFIGURATION

```
# /etc/nginx/sites-enabled/myapp.conf

# Rate limiting zones
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;

# Upstream backends
upstream api_backends {
least_conn;
server app1:8000;
server app2:8000;
keepalive 32;
}

# HTTP -> HTTPS redirect
server {
listen 80;
server_name myapp.com www.myapp.com;
return 301 https://$host$request_uri;
}

# HTTPS server
server {
listen 443 ssl http2;
server_name myapp.com;

# SSL / TLS
ssl_certificate /etc/ssl/myapp.crt;
ssl_certificate_key /etc/ssl/myapp.key;
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers HIGH:!aNULL:!MD5;
ssl_session_cache shared:SSL:10m;

# Security headers
add_header Strict-Transport-Security 'max-age=31536000; includeSubDomains' always;
```

```
add_header X-Frame-Options 'DENY' always;
add_header X-Content-Type-Options 'nosniff' always;
add_header X-XSS-Protection '1; mode=block' always;
add_header Content-Security-Policy "default-src 'self'" always;
```

```
# API proxy
location /api/ {
limit_req zone=api burst=20 nodelay;
proxy_pass http://api_backends;
proxy_http_version 1.1;
proxy_set_header Connection '';
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_read_timeout 30s;
gzip on; gzip_types application/json;
}
```

```
# Static files — served directly by Nginx
location /static/ {
root /var/www;
expires 1y;
add_header Cache-Control 'public, immutable';
}
}
```

INTERVIEW TIP

Always add security headers in Nginx: HSTS (forces HTTPS for 1 year), X-Frame-Options DENY (prevents clickjacking), X-Content-Type-Options nosniff (prevents MIME sniffing), Content-Security-Policy (prevents XSS by controlling which sources can load scripts). These are free security wins.

CHEAT SHEET ANCHOR

Nginx reverse proxy: SSL termination+load balancing+caching+compression+security headers. SSL: TLSv1.2+1.3, strong ciphers. Security headers: HSTS(force HTTPS), X-Frame-Options(no clickjacking), CSP(XSS prevention), nosniff. Rate limiting: limit_req_zone by IP. keepalive for connection reuse.

Q47 What is async programming in Python backends? Explain event loops, coroutines, and FastAPI.

ASYNC I/O IN PYTHON

Python's asyncio enables concurrent I/O operations without threads. One thread handles thousands of concurrent requests by yielding CPU during I/O waits (DB queries, HTTP calls). FastAPI is built on asyncio.

```
import asyncio, httpx, asyncpg

# SYNC version: blocks thread during each I/O operation
def get_user_data_sync(user_id):
user = db.query_sync('SELECT * FROM users WHERE id=%s', user_id)
profile = requests.get(f'http://profile-api/{user_id}').json()
# Total time = db_time + api_time (sequential)

# ASYNC version: both I/O ops run concurrently
async def get_user_data(user_id: int):
async with asyncpg.connect(DATABASE_URL) as conn:
user_task = conn.fetchrow('SELECT * FROM users WHERE id=$1', user_id)
profile_task = httpx.AsyncClient().get(f'http://profile-api/{user_id}')
user, profile = await asyncio.gather(user_task, profile_task)
# Total time = max(db_time, api_time) — concurrent!
return {**dict(user), **profile.json()}
```

FASTAPI ASYNC PATTERNS

```
from fastapi import FastAPI, BackgroundTasks
import asyncio, httpx

app = FastAPI()

# Async endpoint — never blocks thread
@app.get('/users/{user_id}'):
async def get_user(user_id: int, db = Depends(get_db)):
user = await db.fetchrow('SELECT * FROM users WHERE id=$1', user_id)
return dict(user)

# Fan-out: call multiple services concurrently
@app.get('/dashboard/{user_id}')
async def dashboard(user_id: int):
```

```
async with httpx.AsyncClient() as client:
user_req = client.get(f'{USER_SVC}/users/{user_id}')
orders_req = client.get(f'{ORDER_SVC}/users/{user_id}/orders')
notifs_req = client.get(f'{NOTIF_SVC}/users/{user_id}/notifications')
user, orders, notifs = await asyncio.gather(
user_req, orders_req, notifs_req)
return {
'user': user.json(),
'recent_orders': orders.json()[:5],
'notifications': notifs.json()
}
# Background task — don't block response
@app.post('/orders')
async def create_order(order: OrderRequest,
background: BackgroundTasks):
created = await order_service.create(order)
background.add_task(send_confirmation_email, created)
return created # returns before email sent
```

INTERVIEW TIP

asyncio.gather() vs sequential awaits: gather() runs coroutines CONCURRENTLY (total time = max of all). Sequential await runs one after another (total time = sum of all). Always use gather() when multiple I/O operations are independent. Never use time.sleep() in async code — use await asyncio.sleep() instead.

CHEAT SHEET ANCHOR

Async Python: single thread handles many concurrent I/O via event loop. async def+await yields during I/O. asyncio.gather(): concurrent execution of multiple coroutines(total time=max not sum). FastAPI: async endpoints natively. httpx.AsyncClient for async HTTP calls. asyncpg for async PostgreSQL.

Q48 What is the Bulkhead pattern? How do you implement service isolation in Python?

BULKHEAD PATTERN

The Bulkhead pattern isolates different parts of a system into pools so that failure in one part doesn't cascade to others. Named after bulkhead compartments in ships — if one compartment floods, others are unaffected.

- Problem: if Payment Service is slow, all threads waiting for Payment block Order processing too.
- Solution: separate thread pools per service. Payment pool exhausted? Only payment requests fail — orders continue.

BULKHEAD WITH THREAD POOLS

```
from concurrent.futures import ThreadPoolExecutor, TimeoutError
from functools import partial

class BulkheadExecutor:
def __init__(self, name: str, max_workers: int, timeout: float):
self.name = name
self.executor = ThreadPoolExecutor(max_workers=max_workers,
thread_name_prefix=name)
self.timeout = timeout

def execute(self, fn, *args, **kwargs):
future = self.executor.submit(fn, *args, **kwargs)
try:
return future.result(timeout=self.timeout)
except TimeoutError:
future.cancel()
raise TimeoutError(f'{self.name} timed out after {self.timeout}s')

# Separate pools per service
payment_pool = BulkheadExecutor('payment', max_workers=10, timeout=5.0)
inventory_pool = BulkheadExecutor('inventory', max_workers=20, timeout=3.0)
email_pool = BulkheadExecutor('email', max_workers=5, timeout=10.0)

def process_order(order):
try:
# Uses payment pool — exhausted payment doesn't affect inventory
payment = payment_pool.execute(payment_service.charge,
order.payment_details)
```

```
inventory_pool.execute(inventory.reserve,
order.items)
except TimeoutError:
return 'Payment service overloaded, please retry later'
```

INTERVIEW TIP

Bulkhead and Circuit Breaker are the dynamic duo of microservice resilience. Circuit Breaker opens when downstream fails (stops calling it entirely). Bulkhead isolates the threads so the fail doesn't drain your upstream thread pool. Libraries like Resilience4j provide both.

CHEAT SHEET ANCHOR

Bulkhead pattern: isolate failure domains. Thread pool per dependency instead of one giant pool. Payment pool timeout->payment fails, but inventory pool intact so other API endpoints stay healthy. Prevents thread exhaustion cascade.

Q49 What is CQRS and Event Sourcing? Why use them together?

CQRS (Command-Query Responsibility Segregation)

Separates the read model (Query) from the write model (Command). Read DB is optimized for fast queries (NoSQL, Elasticsearch), Write DB is optimized for ACID transactions (PostgreSQL).

EVENT SOURCING

Instead of storing the current state (like UPDATE balance=50), store a sequence of events. The current state is derived by replaying the events. Audit log becomes the source of truth.

```
# Event Sourced Bank Account
events = [
AccountOpened(id=123, balance=0),
Deposited(id=123, amount=100),
Withdrawn(id=123, amount=30),
FeeCharged(id=123, amount=2)
]
# Replay events to get current state
balance = sum([e.amount if type(e)==Deposited else -e.amount for e in events])
```

INTERVIEW TIP

CQRS and Event Sourcing add immense complexity (eventual consistency, two databases, message brokers). Only use them when absolutely necessary (e.g., highly complex domains like accounting, banking, collaborative editing). Don't use them for a simple CRUD app.

CHEAT SHEET ANCHOR

CQRS: split read(NoSQL) from write(SQL). Write syncs to read via async events. Event Sourcing: store state as immutable sequence of events(append-only log), not mutable rows. Audit log IS the DB. Combined: replay events to rebuild read models.

Q50 What are the three pillars of observability? How do you implement Distributed Tracing?

THE THREE PILLARS

- Logs: detailed events (nginx access log, unhandled exceptions).
- Metrics: quantitative numbers (CPU=80%, HTTP 500s=12/sec, Latency=50ms). Used for alerting (Prometheus/Grafana).
- Traces: request lifecycle across multiple services. Find the bottleneck in a microservice chain.

DISTRIBUTED TRACING (OpenTelemetry)

A Trace represents the whole request. A Span represents an operation within the trace. An API Gateway generates a Trace ID, which is passed in HTTP headers (W3C Trace Context) to all downstream services.

```
# Trace Context Header propagation
# traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
# 00 (version) - [Trace ID] - [Parent Span ID] - 01 (sampled flag)
```

```
from opentelemetry import trace
tracer = trace.get_tracer(__name__)
```

```
@app.get('/order/{id}')
def get_order(id: int):
with tracer.start_as_current_span('fetch_order') as span:
span.set_attribute('order.id', id)
order = db.get(id)
if not order:
span.record_exception(ValueError('Not found'))
span.set_status(trace.Status(trace.StatusCode.ERROR))
return order
```

INTERVIEW TIP

OpenTelemetry is now the industry standard, replacing vendor-specific SDKs (Datadog, New Relic) with a vendor-neutral standard. Code is instrumented once with OTel, and the OTel Collector forwards data to any backend.

CHEAT SHEET ANCHOR

Observability: Logs(events), Metrics(numbers, alerts), Traces(request path across services). Span=single operation. Trace=tree of spans. Trace ID propagated via HTTP headers (traceparent) to link spans. OpenTelemetry=vendor neutral telemetry standard.